







中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生数学建模竞赛

学　　校		桂林电子科技大学
参赛队号		**26105950084**
队员姓名	**1.**	常沙
	2.	肖聃
	3.	胡凌峰

中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生数学建模竞赛

题　目： 山区洪涝救援无人机运输与通信协同优化研究

摘　要：

山区洪涝灾害可能同时阻断道路与通信链路，配送面临地形、运力和时限约束。本文以 1 个调度中心、15 个服务区和 80 个不可拆分货箱为对象，依次研究单点往返能力与组批、异构机队多点调度、通信约束下的运输与中继协同，以及冻结任务后的分区配置。各问均需保证规定交付与返航安全；后续调度还需满足医疗及首批保障时限、设备库存和全过程通信要求，并权衡架次数、能耗、完成时间与分组资源代价。

针对问题一，按有向航段飞行剖面分别计算载货去程与空载返程能耗，结合返航安全余量求得连续最大安全载荷；在质量、体积和全架次能量约束下，枚举各服务区不可拆分货箱的可行组批模式。利用能耗随载荷单调增加的性质进行二分计算，再采用记忆化动态规划，按架次数、总能耗和累计作业时间作字典序优化；安全余量改变时重新计算载荷与组批。在 20% 安全余量下，18 架次完成配送，总能耗 59.127 kWh，累计作业时间 546.252 min，达到架次数下界。

针对问题二，建立逐站交付的多点多架次调度模型，按各航段剩余载荷计算时间与能耗，并分别记录实体机和同机型共享电池的占用、返航荷电状态及充满时刻。医疗期望时间和首批截止时间作为硬约束，其他货箱的期望时间用于计算加权延误。先从多个起点构造可行调度，再通过机型调整、架次重排、移箱、换箱、合批与拆批等邻域操作进行模拟退火搜索；资源解码确定无人机、电池及各任务时刻，最终由独立事件回放复核。所得方案以 24 架次完成 80 箱交付，加权延误为 0，最晚返航时间为 103.478 min。

针对问题三，沿运输机爬升、巡航、下降和交接的三维轨迹计算双向链路预算，以直连链路或单架中继的两段可用链路判定通信保障，并约束中继准备、转场、建链、悬停、返航及能源组件周转。地形遮挡与传播损耗共同决定链路状态，运输路线和中继服务窗口共同决定调度可行性。求解时先定位直连盲区并筛选悬停候选，再将中继服务日历纳入运输排序和机型搜索，细化中继任务；独立回放逐段能耗、资源与通信状态，并加密检查低裕量区间。采用 24 个运输架次和 3 个中继架次，联合返航时间为 105.037 min，总能耗为 72.435 kWh。通信结论以离散检查的时空分辨率为限。

针对问题四，固定第三问运输与中继任务的位置和时刻，将同一运输架次涉及的服务区合并为不可拆分块。在组内资源可重新编号、跨组中继完整复制的工作假设下，依据运输机体、电池、中继机体和能源组件的占用区间，计算各组独立执行所需的最小配置及库存缺口。采用并查集建立必须同组的关系，预计算子集需求并枚举无标签分区；区间扫描给出资源峰值下界，区间着色构造相应的组内编号，独立检查任务、充电及跨组资源冲突。两组方案缺 1 架 C 型运输机和 1 架中继机；三组方案缺 1 架 B 型运输机、1 架 C 型运输机、1 组 B 型电池和 2 架中继机。按既定资源目标选择两组方案。

关键词： 无人机运输；运输与通信协同；字典序优化；共享电池调度；任务分区

目录

1 问题重述 6

1.1 问题背景 6

1.2 具体问题 6

2 问题分析与总体思路 7

2.1 问题一 7

2.2 问题二 7

2.3 问题三 7

2.4 问题四 7

2.5 总体解题思路 8

3 模型假设与符号说明 8

3.1 模型假设 8

3.2 符号说明 10

4 数据核验与公共物理模型 12

4.1 数据来源与一致性核验 12

4.2 地形处理与有向航段构建 13

4.2.1 局部坐标与水平距离 13

4.2.2 DEM 登记与节点高度 14

4.2.3 巡航高度与爬升下降 14

4.3 飞行时间与任务时间模型 14

4.3.1 航段飞行时间 14

4.3.2 准备、装载与交接时间 14

4.3.3 不同问题的时间指标 15

4.4 载荷更新、运输能耗与返航安全 15

4.4.1 逐航段载荷更新 15

4.4.2 载荷相关能耗 15

4.4.3 返航 SOC 与安全余量 15

4.5 两阶段充电与资源占用 16

4.5.1 两阶段充电模型 16

4.5.2 机体与能源的独立占用 16

5 问题一：单点往返运输能力与货箱组批 16

5.1 单点往返最大安全载荷模型 16

5.1.1 往返运输能耗 16

5.1.2 安全载荷约束 17

5.1.3 基于单调性的求解方法 17

5.2 货箱组批与机型选择模型 17

5.2.1 可行运输模式构建 17

5.2.2 模式时间与能耗 18

5.2.3 分配约束与字典序目标 18

5.3 完整模式枚举与动态规划求解 18
5.3.1 等价货箱压缩 18
5.3.2 动态规划递推 18
5.4 计算结果与分析 19
5.4.1 最大安全载荷结果 19
5.4.2 货箱组批结果 20
5.4.3 架次数下界与最优性说明 20
5.5 目标优先级对比 21
5.6 返航安全余量敏感性分析 22
5.7 独立核验与适用范围 22
6 问题二：异构无人机多点多架次调度 23
6.1 联合调度模型 23
6.1.1 决策变量与任务表示 23
6.1.2 多点运输的载荷更新 23
6.1.3 逐航段能耗与返航约束 23
6.2 任务时刻与物资时限约束 24
6.2.1 起飞、到达与交付时刻 24
6.2.2 硬时限与软期望时间 24
6.3 无人机与共享电池资源约束 24
6.3.1 机型兼容与实体无人机占用 24
6.3.2 电池预留、充电与复用 25
6.4 多目标优化模型 25
6.5 多起点构造与模拟退火邻域搜索 25
6.5.1 可行方案构造与资源解码 25
6.5.2 邻域与候选接受 25
6.5.3 搜索设置与范围 25
6.6 调度结果与分析 26
6.6.1 基准方案汇总 26
6.6.2 多点架次的载荷与时间示例 27
6.7 目标优先级对比与最优性边界 27
6.7.1 目标优先级比较 27
6.7.2 最优性边界 28
6.8 独立事件回放与可行性检查 28
7 问题三：通信约束下运输与中继联合调度 28
7.1 三维轨迹与通信链路模型 28
7.1.1 运输无人机全过程轨迹 28
7.1.2 双向链路预算 29
7.1.3 自由空间损耗与地形遮挡 29
7.2 中继任务、能耗与资源模型 30
7.2.1 悬停位置与转场剖面 30
7.2.2 有效服务时段 30
7.2.3 中继能耗分解 30
7.2.4 机体与能源组件独立调度 31

7.3 通信保障约束与联合目标 . . . 31
7.3.1 直连与中继保障选择 . . . 31
7.3.2 联合优化目标 . . . 31
7.4 候选生成与联合搜索方法 . . . 31
7.4.1 空间候选与服务结构 . . . 31
7.4.2 通信服务日历参与运输调度 . . . 32
7.4.3 计算限制 . . . 32
7.5 联合调度结果与权衡分析 . . . 32
7.5.1 中继位置与任务安排 . . . 32
7.5.2 联合方案汇总 . . . 33
7.5.3 目标优先级比较 . . . 34
7.6 独立轨迹回放与通信验证 . . . 34
7.6.1 任务与资源检查 . . . 34
7.6.2 分辨率加密检查 . . . 34
7.6.3 验证结论与适用范围 . . . 35
8 问题四：救援任务分区与资源配置优化 . . . 35
8.1 问题分析与建模思路 . . . 35
8.2 冻结方案与解释口径 . . . 36
8.2.1 冻结调度的定义 . . . 36
8.2.2 跨组中继与资源编号假设 . . . 36
8.3 必须同组关系与分区变量 . . . 36
8.4 固定时序下的最小资源配置 . . . 37
8.4.1 资源占用区间 . . . 37
8.4.2 峰值需求与具体资源分配 . . . 37
8.5 分区优化目标 . . . 38
8.5.1 需求、缺口与库存剩余 . . . 38
8.5.2 工作量与字典序目标 . . . 38
8.6 求解方法与最优性范围 . . . 39
8.7 分区与配置结果 . . . 39
8.7.1 两组分区 . . . 39
8.7.2 三组分区 . . . 39
8.8 资源缺口与额外配置的来源 . . . 40
8.8.1 组间错峰复用损失 . . . 40
8.8.2 库存剩余、额外配置与任务重复 . . . 41
8.9 目标优先级与解释口径比较 . . . 41
8.9.1 配置规模与工作量均衡 . . . 41
8.9.2 禁止复制中继架次 . . . 41
8.9.3 继承原实体编号 . . . 42
8.10 独立事件回放与结果有效性 . . . 42
8.11 结果分析 . . . 42
9 模型评价 . . . 42
参考文献 . . . 42

附录A　人工智能产品辅助答题规范说明 **44**
附录B　关键代码展示 **45**

1 问题重述

1.1 问题背景

镇龙乡洪涝救援报道反映了道路受阻、通信受损及应急物资投送需求[1,2]。在此类山区救援任务中，无人机可承担物资运输和空中通信中继，但其运行受载质量、装载体积、地形高差、能源储备及设备数量限制。因此，需要结合物资需求、配送时限和地理环境安排运输与中继任务，并分析任务分区对资源配置的影响。本题以调度中心 O01 及 15 个服务区为研究对象，利用附件中的地理高程、无人机性能、通信链路和物资需求数据，对 80 个不可拆分货箱的配送任务进行建模。研究兼顾运输可行性、配送及时性、能源消耗和通信保障，并在此基础上完成以下四项任务。

1.2 具体问题

问题一: 单点往返运输能力与货箱组批

针对三种运输机型，计算其从 O01 出发、向单个服务区交付货物并空载返回时的最大安全载荷。在满足载质量、装载体积及返航电量要求的条件下，将全部货箱按目的地进行组批，确定各架次的货箱组成和机型，并比较架次数、能耗与累计作业时间之间的关系。

问题二: 异构无人机多点多架次运输调度

允许一个运输架次访问多个服务区，联合确定货箱分配、访问顺序、执行无人机、共享电池及任务时刻。调度方案须满足容量、能源、设备数量和充电复用要求，确保医疗物资与首批保障物资按规定时限完成交付，并综合考虑配送及时性、任务完成时间、能耗和架次数。

问题三: 通信约束下的运输与中继联合调度

在运输调度中引入全过程通信保障要求，根据无人机三维位置、地形遮挡及双向链路条件判断通信可用性。直连网关无法满足要求时，安排中继无人机提供接入与回传保障，联合确定运输方案、中继悬停位置及服务时段，并满足两类无人机及其能源资源的运行约束。

问题四: 救援任务分区与资源配置

以第三问的联合调度方案为基础，分别将 15 个服务区划分为 2 个和 3 个非空任务组。同一运输架次涉及的服务区必须归入同组，并保持既定组批、访问顺序、任务安排和通信保障关系不变。在各组资源不得跨组调配的条件下，核算独立执行所需的运输无人机、共享电池、中继无人机和能源组件，比较不同分区的配置规模、资源冗余与工作量均衡程度，分析相对于现有库存的资源缺口及其原因。

2 问题分析与总体思路

山区洪涝救援中的无人机任务同时受到运输能力、物资时限、能源周转和通信条件的影响。四个问题由单点运输能力分析逐步扩展至多机调度、运输与中继协同，最终研究既定调度下的任务分区与资源配置。各问既具有独立的决策目标，又共享地形、能耗和时间计算规则，因此需要统一基础模型，并明确不同阶段的数据继承关系。

2.1 问题一

问题一需要解决单点往返最大安全载荷计算与货箱组批两个相互关联的问题。最大安全载荷不仅受到额定载质量限制，还取决于运输距离、去返航段的爬升高度以及返航电量下限。由于去程携带货物、返程为空载，两个方向的能耗不能简单按去程能耗加倍计算。同时，连续质量意义上的运输能力并不等同于实际装箱能力，货箱体积和不可拆分性可能进一步限制每架次的装载量。

针对上述特点，首先建立载荷—航程与往返能耗模型，判断空载往返是否可行，再利用能耗随载荷单调变化的性质求取最大安全载荷。在此基础上，按服务区枚举满足质量、体积和能量约束的货箱组合，建立以架次数、总能耗和累计作业时间为字典序目标的组批模型，通过动态规划确定机型与货箱分配方案，并分析安全余量变化对运输能力和组批结果的影响。

2.2 问题二

问题二将单点往返扩展为异构无人机的多点、多架次运输调度，其关键在于货箱组批、访问顺序与有限资源之间的耦合。同一架次经过多个服务区时，货物逐站交付，后续航段的载荷及能耗随之变化；不同架次又竞争实体无人机和共享电池，因此仅优化航线或装载方案，不能保证完整调度可执行。此外，医疗物资和首批保障物资具有硬时限，其他物资的期望时间则用于评价配送及时性，两者需要分别处理。

为处理多点访问造成的载荷变化，本文在统一能耗模型下逐航段更新载荷，并计算交付完成时间。实体无人机和共享电池分别记录占用及恢复可用时刻，使货箱组批、路线与资源安排能够共同接受可行性检查。求解从多个可行起点出发，通过机型调整、批次重排及货箱重组等模拟退火邻域改进方案，以加权延误、最晚运输返航时间、能耗和架次数为字典序目标。最终通过独立事件回放核验货箱、硬时限和资源复用。具体邻域操作在算法部分说明。

2.3 问题三

问题三进一步引入全过程通信约束，使运输路线与中继部署形成时空耦合。运输无人机的通信状态随位置和高度变化，服务区位置可通信并不意味着爬升、巡航、下降和交接阶段均有保障。同时，地形遮挡只是链路损耗的一部分，不能直接将遮挡等同于断联；中继有效服务还要求运输机—中继和中继—网关两段双向链路同时满足条件。

运输机的通信状态取决于实际时刻和三维位置，因此需先由任务安排重建轨迹，再结合地形高程与收发参数识别直连盲区。中继候选位置及其转场、建链、服务、返航和资源周转共同决定保障时段；运输安排也需随之调整。模型综合考虑配送及时性、联合任务完成时间、总能耗和总架次数。求解后对低链路裕量和保障切换附近进行时间、空间加密检查，验证范围在通信验证部分说明。

2.4 问题四

问题四要求在保持第三问任务安排与通信保障关系不变的条件下，将服务区划分为若干独立执行组。其核心不再是重新规划航线，而是分析分区后资源不能跨组复用所造成的配置增加。同一运输架次涉及的服务区必须归入同组，因此应先识别不可拆分的服务区块。对于同时保障不同组的中继任务，还需明确是否允许复制完整任务，以及组内资源是否可

以重新编号，这些解释会直接影响资源需求。

在明确基准口径后，将运输机、中继机、电池和能源组件分别表示为固定时间占用区间，通过最大重叠数确定资源需求下界，并构造相应的组内编号分配方案。在此基础上，对两组和三组的合法分区进行完整枚举，综合比较归一化资源缺口、配置规模与组间工作量均衡程度。同时区分库存剩余、分区造成的额外配置以及中继任务复制带来的重复消耗，并通过独立事件回放核验各组任务和资源分配。

2.5 总体解题思路

如图 2.1 所示, 全文按照基础数据核验—公共物理建模—分层优化求解—独立验证与对比分析的路线展开。首先统一节点坐标、地形高程、货箱需求和设备参数，构建有向航段的距离、爬升下降高度及飞行时间数据，并建立各问共用的载荷、能耗、时间和充电计算规则，避免不同问题之间出现计算口径不一致。

求解从单点运输能力出发：问题一计算安全载荷并优化组批；问题二加入多点访问、物资时限及有限机体和电池，形成可执行的运输调度；问题三在运输任务中加入三维通信判定和中继服务安排。问题四以第三问方案为固定输入，考察分组独立执行引起的资源需求和库存缺口。第一问组批不固定为第二问的约束，第二问方案仅作为第三问的初始候选；只有第四问冻结前序任务。

在结果验证层面，分别采用独立数值复算、资源事件回放和通信分辨率加密检查，核验货箱完整性、硬时限、能源安全及资源使用情况。最后通过安全余量、目标优先级、候选搜索范围和分区解释口径的比较，分析方案变化及其原因，并明确精确枚举、启发式搜索与离散通信检查各自能够支持的结论范围。

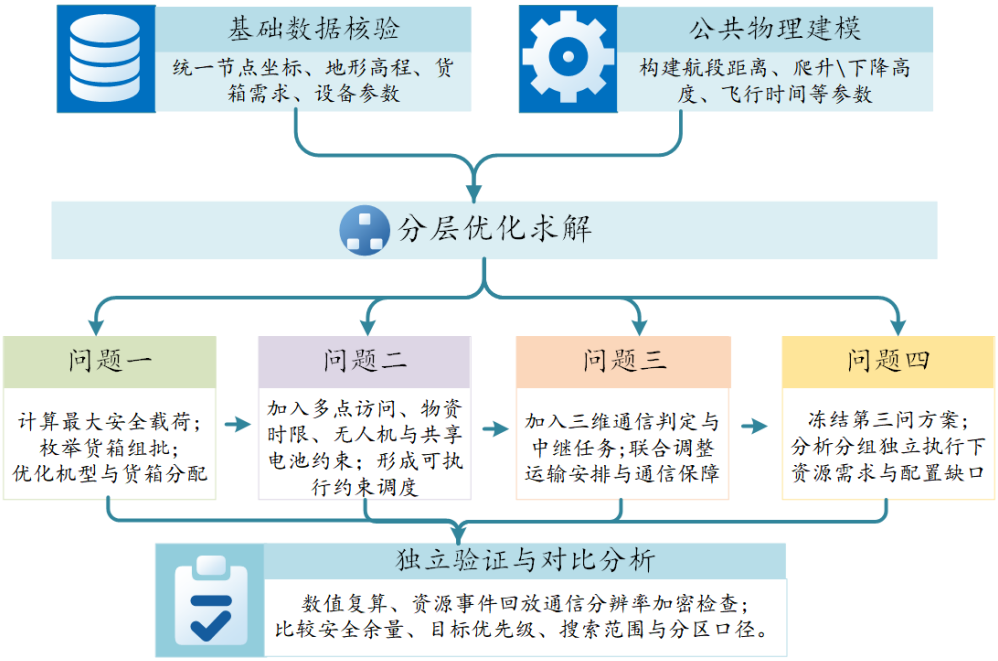


图 2.1　全文建模与求解流程图

3 模型假设与符号说明

3.1 模型假设

为统一四个问题的计算口径，在保留题目规定与附件数据的基础上，对原文未明确的环节作如下建模假设，其适用范围与可能影响在结果分析中说明。货箱不可拆分、医疗与

首批物资的硬时限、设备库存及返航电量下限等属于题目给定约束，不作为另行提出的假设。

1. **给定参数按确定值使用。**以附件中的货箱需求、设备性能和通信参数为基准输入，各航段按给定速度及既定飞行剖面计算。模型不另设随机故障、天气扰动或需求变化，结果对应附件描述的确定性场景。
2. **水平能耗按标准航程比例折算。**假设标准航程对应消耗整组可用电池能量。当机型为 g、航段载荷为 q 时，水平飞行能耗为 $E_{gij}^{\mathrm{hor}}(q)=E_g^{\mathrm{use}}\frac{d_{ij}}{L_g(q)}$，其中，载荷与航程的关系采用题目给定公式 $L_g(q)=L_g^0-\left(L_g^0-L_g^F\right)\left(\frac{q}{Q_g}\right)^{3/2}$. 标准航程的能量标定属于工作假设；若标准航程本身已包含安全储备，则需调整上述折算关系。
3. **爬升效率按能量转化效率解释。**将附件中的爬升效率解释为电能转化为重力势能的效率，爬升附加能耗表示为 $E_{gij}^{\mathrm{climb}}(q)=\frac{(m_g+q)g_0h_{ij}^{+}}{\eta_g\,3.6\times10^6}$，其中 $g_0=9.80665\,\mathrm{m/s^2}$，分母中的换算系数用于将焦耳转换为千瓦时。空载质量使用附件给定口径，不重复加入已包含的电池质量；中继爬升计算使用其计划起飞总质量。
4. **运输交接阶段不另计悬停能耗。**按题目能耗分解计算水平飞行与爬升附加能耗，下降不另计附加能耗。由于缺少运输机交接悬停功率参数，基准模型不额外加入该阶段的悬停能耗。这一简化可能低估实际任务耗电，获得相关功率数据后应重新计算返航安全余量。
5. **交付完成时刻按整次交接结束计算。**将架次开始定义为固定准备开始，完成准备和逐箱装载后起飞。每次服务区交接时长由基础交接时间与逐箱附加时间组成，该次访问的全部货箱统一记为交接结束时完成交付，不以到达时刻代替交付完成时刻。
6. **机体与能源资源分别调度。**运输机体从准备开始占用至返航落地，不额外加入附件未给定的运输周转时间。电池从准备开始预留，返航后可立即拆下充电，不另计拆装时间，再次分配前必须充满。中继机体返航后完成规定周转方可再次使用，能源组件充电可与机体周转并行。不额外设置附件未给定的充电器或准备工位数量限制。
7. **节点作业高度与地形高程分别保留。**节点起降和作业高度采用节点表数据，航段地形与通信遮挡使用 DEM 数据，不通过改写节点海拔消除二者差异。按 PixelIsPoint 登记方式处理像元坐标，在通信判定中将像元内部地形表示为常高表面，射线经过像元边界或角点时采用保守检查。
8. **通信容量与切换采用简化口径。**附件未给出中继并发服务数量、带宽及切换耗时限制时，假设一架中继可同时保障多架满足链路条件的运输机，保障源切换不另计额外时间。每架运输机同一时刻只采用直连或一架中继保障，中继之间不进行多跳转发。该假设用于链路可用性与任务时序计算，实际网络容量仍需设备测试确定。
9. **中继建链功耗与转场剖面采用统一解释。**建链阶段和有效服务阶段均按悬停功率与通信附加功率之和计耗。中继转场采用垂直爬升、水平巡航和垂直下降的分段剖面，转场海拔取 $H^{\mathrm{tr}}=\max\left\{H_{\max}^{\mathrm{DEM}}+50,\,H^{\mathrm{start}},\,H^{\mathrm{end}}\right\}$. 该解释保证转场高度不低于路径净空要求和两端高度，避免出现负爬升或负下降。
10. **第四问允许组内资源重新编号。**冻结第三问基准方案的货箱组批、机型、访问顺序、任务位置、全部时刻及通信保障关系。在此基础上，允许组内同型号机体和能源资源重新编号并合法复用，但不改变任何任务的物理内容，各类实体资源不得跨组调配。同时以继承原实体编号的口径作为对照。
11. **跨组中继采用完整任务副本。**第四问基准假设：当一个原中继架次实际保障多个任务组时，各相关组分别配置该架次的完整副本。副本保留原准备、转场、建链、服务、返航和周转时段，并完整计入资源与能耗；同组内重复引用只计一次。该假设并非题目规定，故另与禁止复制中继的口径比较。复制后的实际干扰、网络容量和空域运行条件未纳入模型。

候选位置离散化、每架次访问点数上限及搜索次数属于算法的计算范围，具体设置见相应章节。第三问的通信结论限定于实际检查的时空分辨率；第四问沿用这一验证范围，连续通信可用性仍未得到数学证明。

3.2 符号说明

为避免不同章节之间出现符号歧义，主要符号统一如下。局部算法状态及辅助变量在对应模型中另行定义。

表 3.1　主要符号及其含义

符号	含义	单位或取值
$\mathcal{S}$	服务区集合，不包含调度中心	集合
0	调度中心 O01 的节点索引	—
i, j	调度中心或服务区的节点索引	—
g	运输无人机机型索引	A, B, C
b	货箱索引	—
k, r	运输架次、中继架次索引	—
w_b, v_b	货箱 b 的质量与体积	kg，m^3
d_b	货箱 b 的期望送达时刻	s
τ_b^{hard}	货箱 b 适用的最早硬截止时刻；无硬时限时取 $+\infty$	s
α_b	货箱 b 的应急优先系数	无量纲
T_b	货箱 b 的交付完成时刻	s
Q_g, V_g	机型 g 的额定载质量与装载体积上限	kg，m^3
m_g	机型 g 按附件口径给出的空载质量	kg
E_g^{use}	机型 g 电池的可用能量	kWh
L_g^0, L_g^F	机型 g 的空载、满载标准航程	m
$L_g(q)$	机型 g 在载荷 q 下的标准航程	m
η_g	机型 g 的爬升效率	无量纲
ρ_g	机型 g 的返航 SOC 下限	比例
q	当前航段携带的货物质量	kg
q_{ig}^*	机型 g 对服务区 i 的最大安全往返载荷	kg
d_{ij}	节点 i 至 j 的水平航段距离	m
h_{ij}^+, h_{ij}^-	有向航段 $i \to j$ 的爬升、下降高度	m
v_g^{cr}	机型 g 的巡航速度	m/s
v_g^+, v_g^-	机型 g 的爬升、下降速度	m/s
$E_{gij}(q)$	机型 g 以载荷 q 执行航段 $i \to j$ 的能耗	kWh
g_0	重力加速度	$\mathrm{m/s}^2$
x_{bk}	货箱 b 是否分配给运输架次 k	$\{0, 1\}$
π_k	运输架次 k 的有序服务区访问序列	序列
$s_k, t_k^{\mathrm{off}}, f_k$	运输架次 k 的准备开始、起飞及返航时刻	s

表 3.1　续表

符号	含义	单位或取值
q_{kij}	运输架次 k 在航段 $i \to j$ 上的剩余载荷	kg
$\mathrm{SOC}_e(t)$	电池或能源组件 e 在时刻 t 的荷电状态	$[0,1]$
T_e^{full}	能源资源 e 的等效完全充电时间	s
τ_e^{chg}	能源资源 e 返航后恢复满电所需时间	s
$\boldsymbol{p}_k(t)$	运输架次 k 在时刻 t 的三维位置	m
$\boldsymbol{p}_r^{\mathrm{hov}}$	中继架次 r 的三维悬停位置	m
c_r, e_r	中继架次 r 的建链完成与服务结束时刻	s
$\mathcal{L}_{uv}(t)$	通信端点 u, v 之间在时刻 t 的总链路损耗	dB
$\mathcal{L}_{uv}^{\max}$	通信端点 u, v 双向链路允许的最大损耗	dB
$M_{uv}(t)$	链路裕量，等于允许损耗减去实际总损耗	dB
$y_{kr}(t)$	时刻 t 运输架次 k 是否由中继架次 r 保障	$\{0,1\}$
D	全部货箱的加权延误	优先系数·s
T_{sum}	第一问各架次累计作业时间之和	s
$C_{\max}$	第二问全部运输无人机的最晚返航时刻	s
C_{joint}	第三问全部运输与中继无人机的最晚返航时刻	s
$E^{\mathrm{T}}, E^{\mathrm{R}}$	运输任务总能耗与中继任务总能耗	kWh
$N^{\mathrm{T}}, N^{\mathrm{R}}$	运输架次数与中继架次数	架次
K	第四问任务组数量	2 或 3
ℓ	任务组索引	$1, \ldots, K$
a	不可拆分服务区块索引	—
$z_{a\ell}$	服务区块 a 是否划入任务组 ℓ	$\{0,1\}$
$c_{r\ell}$	任务组 ℓ 是否需要原中继架次 r 的完整副本	$\{0,1\}$
γ	机体或能源资源类型索引	共 8 类
$n_{\ell\gamma}$	任务组 ℓ 对资源类型 γ 的配置需求	架或组
R_γ, I_γ	资源类型 γ 的分区总需求与现有库存	架或组
Δ_γ, U_γ	资源类型 γ 的配置缺口与库存剩余	架或组
W_ℓ	任务组 ℓ 的机体占用工作量，包含中继周转	s
G, S	归一化资源缺口与归一化配置规模	无量纲
CV	各组工作量的变异系数	无量纲

加权延误仅对具有期望送达时间的货箱计算，具体定义见问题二。医疗物资及首批保障物资的硬时限另由 $T_b \le \tau_b^{\mathrm{hard}}$ 约束，不能通过支付延误惩罚予以放宽。

第四问各类型资源的需求、缺口、剩余及归一化指标在分区目标中统一定义。上述指标用于比较不同类型资源的相对配置规模，不表示货币成本，也不允许用一种资源的剩余抵消另一种资源的缺口。

工作量均衡以各组机体占用工作量的变异系数衡量，具体定义见问题四。需要特别区

分：T_{sum} 是各架次作业时间的累加，$C_{\max}$ 和 C_{joint} 是并行执行下的最晚返航时刻，三者不能直接混用。末次返航后的补电不计入任务完成时间，但仍计入能源资源恢复可用时刻。

4 数据核验与公共物理模型

为保证四个问题的数据一致性与结果可复算性，首先对原始附件、预处理数据及前序计算成果进行核验，并统一航段几何、飞行时间、运输能耗和能源周转的计算规则。后续各问在此基础上加入相应的组批、调度、通信或分区约束，不重复修改公共物理参数。

4.1 数据来源与一致性核验

题目列出的无人机产品规格及无线设备技术资料[3,4]用于说明设备与链路的工程背景，本文计算以题给附件参数和统一规则为准，不以商业产品参数替换附件数据。

基础数据包括调度中心与服务区信息、逐箱物资需求、运输无人机参数、中继无人机参数、通信链路参数及数字高程模型。其中，节点表提供位置与海拔，货箱表提供目的地、质量、体积和时限，设备表提供性能及库存，DEM 用于航段地形净空与通信遮挡分析。

核验后，研究对象包含 1 个调度中心、15 个服务区和 80 个货箱。其中，医疗物资为 16 箱，首批保障物资为 30 箱，两类标记可以重合，不能将其数量直接相加作为具有硬时限的货箱总数。运输及中继资源库存如表 4.2 所示。

表 4.2　无人机与能源资源库存

资源类别	实体无人机数量/架	电池或能源组件数量/组
A 型运输无人机	4	6
B 型运输无人机	2	4
C 型运输无人机	2	4
中继无人机	2	6

共享电池总数已经包含初始机载使用和备用周转所需电池，不能在附件库存之外再次增加初始电池。同机型共享电池可以在不同实体无人机之间复用，不同机型之间不能混用。

数据核验主要包括以下内容：检查货箱编号的唯一性，核对逐箱质量、体积和需求汇总；检查货箱目的地与服务区编号的对应关系；核对设备性能、库存数量及时间单位；检查预处理航段与原始节点数据的一致性。对缺失的期望时间或截止时间保留缺失状态，不将其替换为 0。

对于货箱 b，将其适用的硬截止时间构成集合 $\mathcal{D}_b^{\mathrm{hard}}$。医疗物资的期望送达时间和首批保障物资的首批截止时间分别加入该集合，并定义

$$\tau_b^{\mathrm{hard}} = \begin{cases} \min\limits_{d \in \mathcal{D}_b^{\mathrm{hard}}} d, & \mathcal{D}_b^{\mathrm{hard}} \neq \varnothing, \\ +\infty, & \mathcal{D}_b^{\mathrm{hard}} = \varnothing. \end{cases} \tag{4.1}$$

因此，同一货箱同时具有两类硬时限时，必须满足其中更早的截止时间。普通物资的期望送达时间仅用于计算延误指标，不擅自转为硬约束。

同时，对纳入预处理核验的 9 个原始文件计算 SHA-256，并与已保存的核验记录比较，结果保持一致。经核验的公共数据包含 16 个节点、240 条有向航段，以及三种运输机型对应的 720 条航段飞行时间记录。因此，后续计算复用已有预处理结果，不重复修改原始附件。

4.2 地形处理与有向航段构建

研究区域的地形背景及调度中心、服务区的空间分布如图 4.2 所示。调度中心 O01 与 15 个服务区分布于起伏的山地环境中，节点间的运输能力不仅取决于水平距离，还受到沿途地形高程及起降高度的影响。为统一后续运输与通信模型的空间计算口径，本节依次进行局部坐标转换、DEM 空间登记处理及有向航段飞行剖面构建。图中乡界仅用于说明研究区域，不作为任务分区的约束。

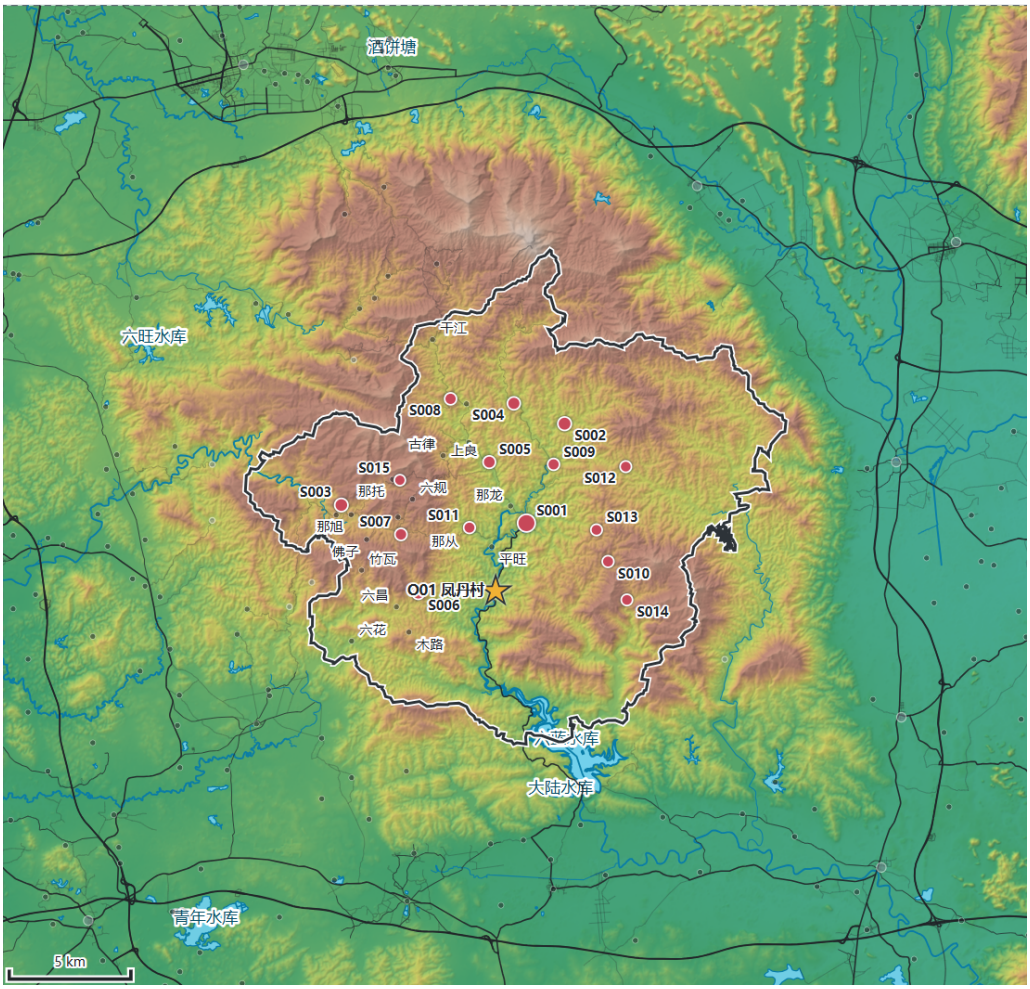


图 4.2　研究区域与节点分布图

注：星形为调度中心 O01，红点为服务区，白色轮廓为镇龙乡边界；底图及节点数据来源于题目附件。

4.2.1 局部坐标与水平距离

为统一距离和能耗计算中的长度单位，将节点经纬度转换为局部米制坐标。设节点 i 的经纬度为 (λ_i, φ_i)，则

$$x_i = k_\lambda(\lambda_i - \lambda_0), \qquad y_i = k_\varphi(\varphi_i - \varphi_0), \tag{4.2}$$

其中，λ_0, φ_0 为局部坐标原点的经纬度，k_λ, k_φ 为经纬度差到米制距离的换算系数。计算中沿用已核验预处理记录中的原点与换算系数，避免各问重新投影产生差异。

节点 i 至 j 的水平距离为 $d_{ij} = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$. 该距离用于水平飞行时间和标准航程比例能耗计算。通信自由空间损耗则使用通信端点之间的三维距离，二者不混用。

4.2.2 DEM 登记与节点高度

题目所列高程数据来源为 Copernicus DEM[5]；本文以配套 GeoTIFF 及其实际地理登记信息为处理依据。

图 4.2 用于展示区域地形与节点位置，具体航段高程和通信遮挡均由原始 DEM 数据计算，不通过图面颜色估读。

原始 GeoTIFF 采用 PixelIsPoint 登记方式，登记坐标对应像元中心。因此，在建立像元边界与空间位置的对应关系时，需考虑半个像元的偏移，避免将中心坐标误作边界坐标。

节点表海拔与 DEM 像元高程并不完全相同，已核验节点的最大绝对差异约为 $10.5629\,\mathrm{m}$。模型保留这一差异：节点起降和作业高度采用节点表，航段地形及通信遮挡采用 DEM，不通过修改节点海拔人为消除地形遮挡。

设节点 i 的表列地面海拔为 z_i，调度中心的作业海拔取 z_0，服务区的作业海拔取 $z_i + 30\,\mathrm{m}$。其中，$i = 0$ 表示调度中心 O01，$\mathcal{S}$ 表示服务区集合，服务区的交接作业高度为地面以上 $30\,\mathrm{m}$。

4.2.3 巡航高度与爬升下降

对运输航段 $i \to j$，沿水平航迹提取穿越地形的最高高程 $z_{ij}^{\mathrm{DEM,max}}$，按题给净空要求确定巡航海拔

$$H_{ij} = z_{ij}^{\mathrm{DEM,max}} + 50. \tag{4.3}$$

相应的爬升和下降高度为

$$h_{ij}^{+} = H_{ij} - z_i^{\mathrm{op}}, \qquad h_{ij}^{-} = H_{ij} - z_j^{\mathrm{op}}. \tag{4.4}$$

上述高度均以米为单位。使用该飞行剖面前，应核验爬升、下降高度的非负性；若端点高度与巡航高度规则发生冲突，应核查剖面，不能直接将负值截断后继续计算。

每次完成服务区交付后，下一航段均从该服务区的作业高度重新爬升。因此，多点运输不能将相邻服务区间的飞行简化为始终保持巡航高度。

往返航段分别记录。即使 $d_{ij} = d_{ji}$，其爬升、下降高度也需按照各自出发端与到达端计算。当爬升和下降速度不同时，正反向飞行时间一般不能直接视为相等。

4.3 飞行时间与任务时间模型

4.3.1 航段飞行时间

设机型 g 的巡航、爬升和下降速度分别为 v_g^{cr}、v_g^{+} 和 v_g^{-}，则航段 $i \to j$ 的飞行时间为

$$t_{gij}^{\mathrm{fly}} = \frac{h_{ij}^{+}}{v_g^{+}} + \frac{d_{ij}}{v_g^{\mathrm{cr}}} + \frac{h_{ij}^{-}}{v_g^{-}}. \tag{4.5}$$

三项依次对应起点垂直爬升、水平巡航和终点垂直下降。

4.3.2 准备、装载与交接时间

设运输架次 k 采用机型 g_k，准备开始时刻为 s_k，装载货箱数为 n_k。若固定准备时间为 $t_{g_k}^{\mathrm{prep}}$，每箱装载时间为 $t_{g_k}^{\mathrm{load}}$，则起飞时刻为 $t_k^{\mathrm{off}} = s_k + t_{g_k}^{\mathrm{prep}} + n_k t_{g_k}^{\mathrm{load}}$.

设架次 k 在服务区 i 交付 n_{ki} 个货箱，基础交接时间为 $t_{g_k}^{\mathrm{base}}$，每箱附加交接时间为 $t_{g_k}^{\mathrm{box}}$，则该次访问的交接时长为 $t_{ki}^{\mathrm{service}} = t_{g_k}^{\mathrm{base}} + n_{ki} t_{g_k}^{\mathrm{box}}$. 若到达时刻为 a_{ki}，则本次交付的每个货箱 b 均满足 $T_b = a_{ki} + t_{ki}^{\mathrm{service}}$.

沿访问顺序递推各航段飞行和服务时间，可得到架次返航时刻 f_k：

$$f_k = t_k^{\mathrm{off}} + \sum_{(i,j)\in\mathcal{A}_k} t_{g_k ij}^{\mathrm{fly}} + \sum_{i\in\pi_k} t_{ki}^{\mathrm{service}}, \tag{4.6}$$

其中，$\mathcal{A}_k$ 包含从 O01 出发、访问服务区及返回 O01 的全部航段。

4.3.3 不同问题的时间指标

第一问的累计作业时间为 $T_{\text{sum}} = \sum_k (f_k - s_k)$, 其含义是各架次准备、装载、飞行与交接时间之和，不包含充电和排队，也不等于多机并行执行后的任务历时。

第二问以所有运输机的最晚返航时刻作为完成时间：$C_{\max} = \max_k f_k$.

第三问同时考虑运输和中继任务：$C_{\text{joint}} = \max\{\max_k f_k, \max_r f_r\}$. 其中 f_r 为中继架次 r 的返航时刻。末次返航后的补电及中继机体周转不计入联合完成时间，但仍需在资源状态中记录。

4.4 载荷更新、运输能耗与返航安全

4.4.1 逐航段载荷更新

设 $\mathcal{B}_k$ 为分配给架次 k 的货箱集合，$\mathcal{D}_{ki}$ 为其从节点 i 出发前已经交付的货箱集合，则当前航段的剩余载荷为 $q_{kij} = \sum_{b \in \mathcal{B}_k \setminus \mathcal{D}_{ki}} w_b$. 剩余货物体积同理为 $V^{\text{load}}_{kij} = \sum_{b \in \mathcal{B}_k \setminus \mathcal{D}_{ki}} v_b$. 各航段应满足 $0 \le q_{kij} \le Q_{g_k}, \qquad 0 \le V^{\text{load}}_{kij} \le V_{g_k}$. 全部货物交付后，返回 O01 的航段载荷为 0。

4.4.2 载荷相关能耗

无人机配送研究将载荷与能耗的关系视为路径规划的重要约束，不同能耗模型也有各自的适用条件[6,7]。本文以下计算采用题给载荷–航程关系，并按第 3 节所述的能量标定假设确定航段能耗。

采用题给载荷与航程关系

$$L_g(q) = L_g^0 - (L_g^0 - L_g^F)\left(\frac{q}{Q_g}\right)^{3/2}. \tag{4.7}$$

在前述标准航程能量标定和爬升效率工作假设下，运输机执行航段 $i \to j$ 的能耗为

$$E_{gij}(q) = E_g^{\text{use}} \frac{d_{ij}}{L_g(q)} + \frac{(m_g + q) g_0 h_{ij}^{+}}{\eta_g \, 3.6 \times 10^6}. \tag{4.8}$$

第一项为水平飞行能耗，第二项为爬升附加能耗。距离统一使用米，质量使用千克，能量结果为千瓦时。

架次 k 的运输能耗按实际剩余载荷逐段累计：

$$E_k = \sum_{(i,j) \in \mathcal{A}_k} E_{g_k ij}(q_{kij}). \tag{4.9}$$

因此，不能用起飞载荷计算全部航段，也不能用单程能耗乘以 2 代替载货去程与空载返程。

中继无人机的水平飞行采用功率与时间相乘计算，不套用运输机的载荷—航程标定式。其中继爬升、建链及悬停通信能耗在第三问中分别建立。

4.4.3 返航 SOC 与安全余量

能源初始为满电时，运输架次 k 的返航 SOC 为 $\text{SOC}_k^{\text{return}} = 1 - \frac{E_k}{E_{g_k}^{\text{use}}}$. 返航安全约束表示为 $\text{SOC}_k^{\text{return}} \ge \rho_{g_k}$, 等价地，

$$E_k \le (1 - \rho_{g_k}) E_{g_k}^{\text{use}}. \tag{4.10}$$

基准安全余量按附件取值，敏感性分析中的其他余量仅作为对比情景。对于空载往返已不满足能量约束的情况，应标记为不可行，不能用最大安全载荷为 0 掩盖不可达性。

4.5 两阶段充电与资源占用

4.5.1 两阶段充电模型

锂离子电池充电具有恒流与恒压等阶段特征[8]。本文的分段阈值与时间比例采用题目给定规则，不直接由该技术资料推定。

设能源资源 e 的返航 SOC 为 s，等效完全充电时间为 T_e^{full}。根据题给两阶段规则，恢复至满电所需时间为

$$\tau_e^{\text{chg}}(s) = \begin{cases} T_e^{\text{full}} \left[0.65 \dfrac{0.9 - s}{0.9} + 0.35 \right], & 0 \le s < 0.9, \\ T_e^{\text{full}} \cdot 0.35 \dfrac{1 - s}{0.1}, & 0.9 \le s \le 1. \end{cases} \tag{4.11}$$

该式适用于运输电池和中继能源组件，但其完全充电时间分别读取相应机型参数。

沿用返航后立即开始充电的工作假设，若任务返航时刻为 f，则能源恢复可用时刻为 $t_e^{\text{ready}} = f + \tau_e^{\text{chg}}(s)$. 下一次使用该能源的准备开始时刻不得早于 t_e^{ready}。

4.5.2 机体与能源的独立占用

运输机体和电池的可用时刻通常不同，因此将其作为独立资源记录。各类资源的占用区间统一定义为

$$\mathcal{I}_k^{\text{T,body}} = [s_k, f_k), \tag{4.12}$$

$$\mathcal{I}_k^{\text{T,battery}} = [s_k, t_{e(k)}^{\text{ready}}), \tag{4.13}$$

$$\mathcal{I}_r^{\text{R,body}} = [s_r, f_r + t^{\text{turn}}), \tag{4.14}$$

$$\mathcal{I}_r^{\text{R,energy}} = [s_r, t_{e(r)}^{\text{ready}}), \tag{4.15}$$

其中，$e(k)$ 和 $e(r)$ 分别表示任务使用的电池或能源组件，t^{turn} 为中继机体规定的周转时间。

同一实体资源的占用区间不得重叠。区间采用左闭右开的形式，在严格同一时刻完成释放并开始下一任务时，先释放后占用。计算使用未四舍五入的数据，展示精度不参与可行性判断。独立复算的时间、能量和 SOC 容差分别取 10^{-7} s、10^{-8} kWh 和 10^{-10}。

第一问仅使用上述公共时间与能耗规则，不引入实体设备库存和充电调度约束；第二、三问将库存作为硬约束；第四问则冻结第三问任务时序，按组独立核算资源需求，并将超过现有库存的部分明确列为配置缺口。这一处理保证各问共享物理基础，同时保留各自不同的研究范围。

5 问题一：单点往返运输能力与货箱组批

本问研究运输无人机执行 O01 $\to S_i \to$ O01 单点往返任务时的运输能力与货箱分配。每架次仅向一个服务区配送，去程携带本架次全部货物，交付后空载返回调度中心。首先计算各机型面向不同服务区的最大安全载荷，再结合货箱体积及不可拆分要求确定组批和机型选择。

本问不引入实体无人机数量、共享电池调度、充电、通信及后续问题的配送时限约束。能耗计算沿用公共物理模型中的工作假设，所得结论限定于该能量标定与计算口径。

5.1 单点往返最大安全载荷模型

5.1.1 往返运输能耗

设机型 g 携带质量为 q 的货物，从 O01 前往服务区 i。载荷—航程关系沿用公共物理模型。去程载荷为 q，返程载荷为 0，两个方向分别采用对应航段的距离和爬升高度。在标

准航程能量标定及爬升效率解释的工作假设下，全架次往返能耗为

$$
\begin{aligned}
E^{\mathrm{rt}}_{gi}(q) = E^{\mathrm{use}}_{g}\frac{d_{0i}}{L_g(q)} + \frac{(m_g+q)g_0h^{+}_{0i}}{\eta_g\, 3.6\times 10^{6}} \\
+ E^{\mathrm{use}}_{g}\frac{d_{i0}}{L^{0}_{g}} + \frac{m_g g_0 h^{+}_{i0}}{\eta_g\, 3.6\times 10^{6}}.
\end{aligned}
\tag{5.16}
$$

式 (5.16) 中的四项依次为去程水平飞行能耗、去程爬升附加能耗、空载返程水平飞行能耗和空载返程爬升附加能耗。其中 m_g 已经按附件口径包含电池质量，不再重复加入电池质量。

返程虽然不携带货物，但仍需从服务区作业高度爬升至巡航高度，因此不能忽略返程爬升能耗。下降不另计附加能耗，运输交接阶段未额外加入缺少功率参数的悬停能耗。

5.1.2 安全载荷约束

设机型 g 的返航 SOC 下限为 ρ_g，则全架次允许消耗的能量上限为 $B_g = (1-\rho_g)E^{\mathrm{use}}_{g}$. 最大安全载荷定义为

$$
\begin{aligned}
q^{*}_{ig} = \max_{q} \quad & q, \\
\text{s.t.} \quad & 0 \le q \le Q_g, \\
& E^{\mathrm{rt}}_{gi}(q) \le B_g.
\end{aligned}
\tag{5.17}
$$

式 (5.17) 同时考虑额定载质量和全架次返航安全余量。它描述的是连续质量意义上的运输能力，尚未包含具体货箱的体积和不可拆分限制。实际可装货物还需在组批模型中进一步检验。

求解前先判断空载往返是否可行。若 $E^{\mathrm{rt}}_{gi}(0) > B_g$，则该机型无法在规定安全余量下完成该服务区的往返任务，应标记为不可行，不能将其解释为最大载荷为 0。

5.1.3 基于单调性的求解方法

由附件参数可知 $L^{0}_{g} > L^{F}_{g} > 0$，且因此，水平能耗随载荷增加而增大，爬升附加能耗也随载荷线性增加。在本数据对应的非负爬升条件下，$E^{\mathrm{rt}}_{gi}(q)$ 关于 q 单调不减。

据此，先检查空载与满载往返可行性；仅在两者之间存在能耗边界时进行二分。边界情况下，使用二分法搜索能量约束对应的载荷边界。每次保留满足约束的一侧，最终输出可行侧的载荷值。程序采用 80 轮二分，实际精度受浮点运算精度限制，可行性判断使用未四舍五入数据。

5.2 货箱组批与机型选择模型

5.2.1 可行运输模式构建

设 $\mathcal{B}_i$ 为目的地为服务区 i 的货箱集合。一个运输模式 k 由服务区 i、机型 $g(k)$ 以及非空货箱子集 $\mathcal{B}_k \subseteq \mathcal{B}_i$ 组成。

该模式的箱数、总质量和总体积分别为 $n_k = |\mathcal{B}_k|$, $q_k = \sum_{b\in\mathcal{B}_k} w_b$, $v_k = \sum_{b\in\mathcal{B}_k} v_b$.

仅保留满足以下约束的模式：

$$
q_k \le Q_{g(k)}, \tag{5.18}
$$

$$
v_k \le V_{g(k)}, \tag{5.19}
$$

$$
E^{\mathrm{rt}}_{g(k)i}(q_k) \le (1-\rho_{g(k)})E^{\mathrm{use}}_{g(k)}. \tag{5.20}
$$

记服务区 i 的全部可行模式集合为 $\mathcal{P}_i$。

由于模式按服务区分别生成，其内部已经满足不得跨服务区组批的要求；模式中的每个货箱均以完整个体加入，不允许拆分。

5.2.2 模式时间与能耗

可行模式 k 的能耗为 $e_k = E^{\mathrm{rt}}_{g(k)i}(q_k)$. 其完整作业时间为

$$\begin{aligned}\tau_k = t^{\mathrm{prep}}_{g(k)} + n_k t^{\mathrm{load}}_{g(k)} + t^{\mathrm{fly}}_{g(k),0i} + t^{\mathrm{fly}}_{g(k),i0} \\ + t^{\mathrm{base}}_{g(k)} + n_k t^{\mathrm{box}}_{g(k)}.\end{aligned} \tag{5.21}$$

式 (5.21) 包含固定准备、逐箱装载、往返飞行以及服务区交接时间，不包含充电和排队。其中，往返纯飞行时间仅为 $t^{\mathrm{fly}}_{g(k),0i} + t^{\mathrm{fly}}_{g(k),i0}$，不能与完整作业时间混用。

5.2.3 分配约束与字典序目标

在逐箱模式表示下，定义二元决策变量令 $a_{bk} = 1$ 表示模式 k 包含货箱 b，否则 $a_{bk} = 0$。为保证每个货箱恰好配送一次，建立约束

$$\sum_{k \in \mathcal{P}_i} a_{bk} y_k = 1, \qquad \forall b \in \mathcal{B}_i, \quad \forall i \in \mathcal{S}. \tag{5.22}$$

总架次数 N、总能耗 E 和累计作业时间 T_{sum} 分别对选中模式的相应指标求和。基准采用字典序目标

$$\operatorname{lex\,min} \left(N, E, T_{\mathrm{sum}}\right). \tag{5.23}$$

即首先最小化总架次数，在架次数最少的方案中最小化能耗，再在前两项最优的方案中最小化累计作业时间。该优先级属于建模选择，不是题目指定的唯一排序。

5.3 完整模式枚举与动态规划求解

5.3.1 等价货箱压缩

本问不引入物资类别对应的配送时限，同一服务区内质量和体积均相同的货箱，对容量、能耗和作业时间具有相同影响。因此，将这些货箱合并为等价类，以各类剩余箱数代替逐箱二元状态。

设服务区 i 共有 J_i 类等价货箱，剩余需求状态为 $\boldsymbol{r} = (r_1, r_2, \ldots, r_{J_i})$. 模式 k 用箱数向量 $\boldsymbol{a}_k = (a_{1k}, a_{2k}, \ldots, a_{J_i k})$ 表示，各分量为该模式装载的对应类别箱数。模式求解完成后，再将等价类中的原始货箱编号逐一分配回架次。

该压缩仅用于第一问，在当前约束和目标下不丢失信息；后续引入逐箱时限后，不能未经检验直接沿用相同等价关系。

5.3.2 动态规划递推

依据动态规划的最优子结构思想[9]，将剩余货箱状态与可行运输模式相结合，建立如下递推关系。

定义 $F_i(\boldsymbol{r})$ 为完成剩余需求 $\boldsymbol{r}$ 所需的最优成本三元组。边界条件为 $F_i(\mathbf{0}) = (0, 0, 0)$.

对非空状态，取第一个剩余箱数为正的类别 $j^* = \min\{j : r_j > 0\}$. 在所有包含该类别、且不超过剩余需求的可行模式中递推：

$$F_i(\boldsymbol{r}) = \operatorname{lex\,min}_{\substack{k \in \mathcal{P}_i \\ \boldsymbol{a}_k \le \boldsymbol{r} \\ a_{j^* k} \ge 1}} \left[(1, e_k, \tau_k) + F_i(\boldsymbol{r} - \boldsymbol{a}_k)\right]. \tag{5.24}$$

其中，向量加法按分量进行，$\boldsymbol{a}_k \le \boldsymbol{r}$ 也按分量比较。若不存在满足条件的转移，则该状态不可行。

任一完整分组方案中，必然存在至少一个模式包含类别 j^{*}，因此上述锚定方式不会遗漏合法方案，仅减少同一组批方案因架次排列不同产生的重复搜索。

各服务区之间不存在设备数量、充电或时间资源耦合，三个目标又均为可加指标，因此可分别求解各服务区，再汇总得到总体方案。模式枚举与状态转移均完整执行，未采用超时截断或启发式剪枝。

5.4 计算结果与分析

5.4.1 最大安全载荷结果

以附件规定的 20% 返航安全余量为基准，计算三种机型对 15 个服务区的单点往返最大安全载荷，共得到 45 项结果，如表 5.3 所示。各组合均满足空载往返条件，表中载荷为额定载质量与往返能量约束共同作用下的连续质量上限，具体货箱组批还需满足体积与不可拆分约束。

表 5.3　各机型单点往返最大安全载荷

服务区	A 型/kg	B 型/kg	C 型/kg
S001	25.000000	30.000000	80.000000
S002	25.000000	30.000000	68.861224
S003	25.000000	30.000000	68.217586
S004	25.000000	30.000000	63.698047
S005	25.000000	30.000000	80.000000
S006	25.000000	30.000000	80.000000
S007	25.000000	30.000000	80.000000
S008	25.000000	28.801769	58.905298
S009	25.000000	30.000000	80.000000
S010	25.000000	30.000000	80.000000
S011	25.000000	30.000000	80.000000
S012	25.000000	30.000000	68.122153
S013	25.000000	30.000000	80.000000
S014	25.000000	30.000000	80.000000
S015	25.000000	30.000000	80.000000

为便于比较不同机型，将最大安全载荷除以相应额定载质量，结果绘于图 5.3。该比例描述指定航线上的安全载荷能力，实际组批装载情况另见架次结果。表中数值及图中标注仅用于展示，可行性判断均使用未舍入结果。

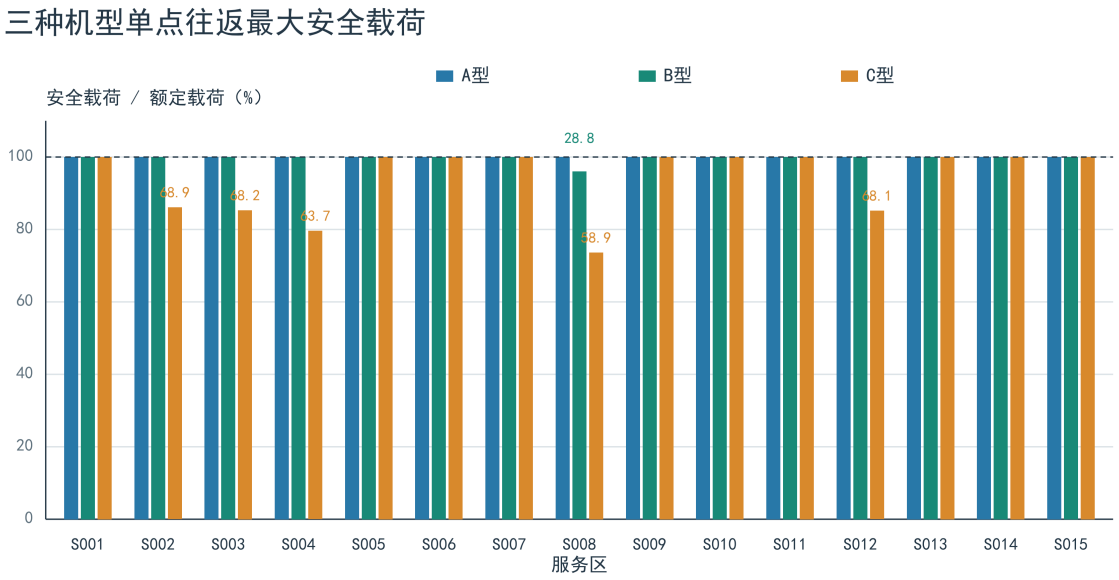


图 5.3　三种机型单点往返最大安全载荷占额定载荷的比例

注：虚线表示 100% 额定载荷；受限柱上的数值为实际最大安全载荷，单位为 kg。返航安全余量取 20%。

结合表 5.3 与图 5.3 可知，A 型在全部服务区均可达到额定载质量 25 kg，其载荷上限由额定载质量决定。B 型仅在 S008 受到往返能量约束，最大安全载荷约为 28.802 kg，相当于额定载质量的 96.01%。

C 型在 S002、S003、S004、S008 和 S012 受到能量约束，其余服务区均可达到额定载质量 80 kg。其中，S008 对应的 C 型最大安全载荷最低，约为 58.905 kg，仅为额定载质量的 73.63%，载荷能力受限最为明显。

S008 等航线的能量约束先于额定载质量约束生效。载荷增加会缩短可用航程并提高爬升能耗，航段距离和地形高差又放大了这一影响。因此，货箱组批需采用各机型在对应服务区的安全载荷上限，而非统一按额定载质量装载。

5.4.2 货箱组批结果

按照式 (5.23) 求解，全部 80 个货箱共分为 18 个运输架次，其中 B 型 9 架次、C 型 9 架次，未选用 A 型。架次摘要如表 5.4 所示，原始货箱编号的完整对应关系保存在结果附件中。

基准方案共 18 架次，总能耗为 59.127133652 kWh，累计作业时间为 32775.128089 s，约 546.252 min。累计作业时间是各架次完整作业时间之和，与多机并行执行的总历时不同。B 型和 C 型各执行 9 架次；本问尚未设置实体机数量约束，因此此处的架次数不作为两种机型的实体机配置数。

方案中最低返航 SOC 为 23.089108%，对应 S004 的 C 型运输架次，高于基准下限 20% 约 3.089108 个百分点。在本组需求、参数与目标排序下，基准方案未选用 A 型。

5.4.3 架次数下界与最优性说明

S001 的货物总质量为 154 kg，S002 和 S003 均为 81 kg。三种机型中最大的额定载质量为 80 kg，因此这三个服务区均至少需要 2 架次。其余 12 个服务区各有配送需求，至少需

表 5.4　第一问基准货箱组批方案

架次	服务区	机型	箱数	质量/kg	体积/m^3	能耗/kWh
P001	S001	C	7	76	0.159	2.917477
P002	S001	C	8	78	0.235	2.995661
P003	S002	B	3	25	0.067	2.713386
P004	S002	C	5	56	0.144	5.721171
P005	S003	B	3	25	0.067	2.779588
P006	S003	C	5	56	0.144	5.764319
P007	S004	C	6	59	0.156	6.152871
P008	S005	C	6	59	0.156	4.222152
P009	S006	C	6	59	0.156	2.597321
P010	S007	C	5	45	0.129	3.410417
P011	S008	C	5	45	0.129	5.835969
P012	S009	B	3	25	0.067	2.096184
P013	S010	B	3	25	0.067	1.822820
P014	S011	B	3	25	0.067	1.073904
P015	S012	B	3	25	0.067	2.752261
P016	S013	B	3	25	0.067	1.824515
P017	S014	B	3	25	0.067	2.139813
P018	S015	B	3	25	0.067	2.307304

要 1 架次，从而得到总架次数下界

$$N \geq \left\lceil \frac{154}{80} \right\rceil + \left\lceil \frac{81}{80} \right\rceil + \left\lceil \frac{81}{80} \right\rceil + 12 = 18. \tag{5.25}$$

实际方案满足全部质量、体积和能量约束，并达到 18 架次，因而第一层目标已达到该下界。在最少架次数条件下，完整模式枚举与动态规划进一步比较能耗和累计作业时间，得到给定工作假设、航段数据及数值精度下的全局字典序最优方案。

该下界依赖第一问不得跨服务区组批的条件，不能直接作为第二问多点运输的架次数下界。

5.5 目标优先级对比

为分析架次数、能耗与时间之间的权衡，分别采用能耗优先和累计作业时间优先的目标排序重新求解，结果如表 5.5 所示。

表 5.5　不同目标优先级下的组批结果

字典序目标	架次数	能耗/kWh	累计作业时间/s
$N \to E \to T_{\text{sum}}$	18	59.127134	32775.128089
$E \to N \to T_{\text{sum}}$	19	59.029753	34439.090232
$T_{\text{sum}} \to N \to E$	18	59.127134	32775.128089

能耗优先方案比基准增加 1 架次，总能耗减少约 0.097381 kWh，降幅约为 0.1647%，但累计作业时间增加约 1663.962 s，即 27.733 min。累计作业时间优先方案在本数据下与基准

指标相同。

因此，本算例中进一步降低能耗需要付出额外架次和作业时间代价。基准选择架次数优先，是对减少运输任务数量的偏好，并非认定该优先级在所有救援场景中均为唯一合理选择。

5.6 返航安全余量敏感性分析

设共同返航安全余量为 ρ，则各机型、服务区的单点往返任务须满足 $E_{g,\mathrm{round}}(q) \le (1-\rho)E_g^{\mathrm{use}}$. 以附件规定的 20% 为基准，选取 10%、15%、25%、30% 和 40% 作为对比情景。每个情景均重新计算三种机型对 15 个服务区的 45 项连续最大安全载荷，并重新筛选货箱可行组合、求解 $N \to E \to T_{\mathrm{sum}}$ 字典序组批；因此表 5.6 中的架次数与能耗不是对基准方案的事后检查。

表 5.6　返航安全余量变化下的安全载荷与组批结果

ρ/%	S004 的 C 型/kg	S008 的 C 型/kg	架次数	能耗/kWh	累计作业时间/min
10	74.885	71.439	18	59.127	546.252
15	69.914	65.903	18	59.127	546.252
20	63.698	58.905	18	59.127	546.252
25	55.663	49.706	19	61.069	576.080
30	44.749	36.815	20	67.223	609.755
40	—	—	—	—	—

表中安全载荷为连续质量能力，实际组批仍需逐箱满足体积和不可拆分约束；“—”表示该机型在相应服务区连空载往返也不满足能量要求，不能解释为零载荷可运输。从 10% 至 20%，最优组批均为 18 架次、59.127133652 kWh，累计作业时间为 32775.128089 s；但 S004 和 S008 的 C 型最大安全载荷仍随余量提高而持续下降。可见连续载荷能力的变化并不必然立即改变离散货箱组批。

以基准架次数为目标，逐服务区枚举组批并最大化最低返航 SOC，保持 18 架次的最大共同安全余量为 23.0891084%，瓶颈位于 S004。在该阈值下方和上方各 10^{-7} 的余量处重新求解，分别得到 18 架次与 19 架次，而不是仅将基准方案代入新余量作可行性检查。当 $\rho = 25\%$ 时，结果为 19 架次、61.069187920 kWh；S008 保持单架次的最大余量为 27.0503919%，所以进一步提高到 30% 时，结果增至 20 架次、67.223154636 kWh。上述数值阈值只针对本题附件和已说明的能耗工作假设。

当余量提高至 40%，S004 和 S008 已有货箱无法由任何机型按单点往返方式送达，完整 80 箱组批在本问约束下不可行；表中因此不填写完整方案的架次数、能耗或时间。情景文件还记录了不可行货箱编号。安全余量提高会降低可承载质量，并可通过拆分组批增加任务量；达到单箱也不可运输的条件时，仅增加架次数已无法恢复可行性。

5.7 独立核验与适用范围

使用独立于组批求解程序的检查程序，从原始货箱数据重新计算各架次的质量、体积、去程载荷、空载返程能耗、返航 SOC 及完整作业时间。核验结果表明，80 个货箱均恰好分配一次，目的地正确，18 个架次均满足载质量、体积和返航安全余量要求，汇总能耗与时间同求解结果一致。

第一问的最优性结论适用于上述确定性模型。标准航程能量标定、爬升效率解释及未计入运输交接悬停能耗的简化，会影响实际飞行能耗与安全载荷；采用其他能耗解释或环境条件时，方案需重新计算。安全余量变化下的载荷和组批结果见表 5.6。

6 问题二：异构无人机多点多架次调度

问题二允许运输无人机在同一架次中访问多个服务区，并引入实体无人机、共享电池、充电周转及物资配送时限约束。因此，需要联合确定货箱组批、访问顺序、执行机型、实体无人机、电池编号和任务开始时刻。

本问继承第一问的航段时间与运输能耗计算规则，但不固定第一问的货箱组批及机型选择。由于多点运输与有限资源调度改变了可行域，第一问的 18 架次下界不能直接用于本问。

6.1 联合调度模型

6.1.1 决策变量与任务表示

设 $\mathcal{B}$ 为全部货箱集合，$\mathcal{U}_g$ 和 $\mathcal{E}_g$ 分别为机型 g 对应的实体无人机集合和共享电池集合。附件给定 A/B/C 型运输机分别为 4/2/2 架，共享电池分别为 6/4/4 组。对运输架次 k，主要决策包括：货箱集合 $\mathcal{B}_k$、执行机型 g_k、实体无人机 u_k、电池 e_k、准备开始时刻 s_k 以及访问序列 $\pi_k = (i_{k1}, i_{k2}, \ldots, i_{km_k})$, 其中 m_k 为该架次的服务区访问次数。每架次均从 O01 出发，并在完成交付后返回 O01。

定义 $x_{bk} \in \{0, 1\}$ 表示货箱 b 是否分配给架次 k，则所有货箱满足

$$\sum_{k} x_{bk} = 1, \qquad \forall b \in \mathcal{B}. \tag{6.26}$$

同时，所分配货箱的目的地必须出现在对应架次的访问序列中，并且只能在其正确目的地完成交付。

6.1.2 多点运输的载荷更新

设 $\mathcal{B}_{k\ell}$ 为架次 k 在第 ℓ 次访问时交付的货箱集合。各交付集合互不重叠，且满足 $\bigcup_{\ell=1}^{m_k} \mathcal{B}_{k\ell} = \mathcal{B}_k$.

起飞时的货物质量与体积分别为 $q_{k0} = \sum_{b\in\mathcal{B}_k} w_b, \qquad V_{k0}^{\text{load}} = \sum_{b\in\mathcal{B}_k} v_b$. 完成第 ℓ 次交付后，剩余载荷更新为

$$q_{k\ell} = q_{k,\ell-1} - \sum_{b\in\mathcal{B}_{k\ell}} w_b, \tag{6.27}$$

$$V_{k\ell}^{\text{load}} = V_{k,\ell-1}^{\text{load}} - \sum_{b\in\mathcal{B}_{k\ell}} v_b. \tag{6.28}$$

因此，所有交付完成后应有 $q_{km_k} = 0, \qquad V_{km_k}^{\text{load}} = 0.$

各航段均需满足载质量与体积限制：

$$0 \le q_{k\ell} \le Q_{g_k}, \qquad 0 \le V_{k\ell}^{\text{load}} \le V_{g_k}. \tag{6.29}$$

由于本问只交付货物而不沿途取货，载荷和体积随交付单调减少；但在能耗计算中，仍需逐航段使用对应的实际剩余载荷。

6.1.3 逐航段能耗与返航约束

记 $i_{k0} = i_{k,m_k+1} = 0$。架次 k 的总能耗为

$$E_k = \sum_{\ell=0}^{m_k} E_{g_k, i_{k\ell}, i_{k,\ell+1}}(q_{k\ell}), \tag{6.30}$$

其中，航段能耗函数沿用第四节公共物理模型。每次服务区交付后，下一航段均从该点地面以上 30 m 作业高度重新爬升，再完成巡航与下降。不能将多点访问视为全程保持固定巡航高度，也不能用起飞载荷计算后续全部航段的能耗。

全架次返航安全约束为 $E_k \le (1-\rho_{g_k})E^{\text{use}}_{g_k}$，对应返航 SOC 为 $1-E_k/E^{\text{use}}_{g_k}$，不低于安全余量 ρ_{g_k}。上述能耗表达仍依赖已说明的能量标定、爬升效率解释及未额外计入运输交接悬停能耗等工作假设。

6.2 任务时刻与物资时限约束

6.2.1 起飞、到达与交付时刻

设架次 k 装载 n_k 个货箱，起飞时刻按准备、逐箱装载与固定准备开始时刻相加，沿用第四节的统一时间模型。以 $a_{k\ell}$ 和 $c_{k\ell}$ 分别表示第 ℓ 次访问的到达时刻和交接完成时刻。首个服务区的到达时刻为 $a_{k1} = t^{\text{off}}_k + t^{\text{fly}}_{g_k,0,i_{k1}}$. 之后按访问顺序递推：

$$c_{k\ell} = a_{k\ell} + t^{\text{base}}_{g_k} + |\mathcal{B}_{k\ell}|t^{\text{box}}_{g_k}, \tag{6.31}$$

$$a_{k,\ell+1} = c_{k\ell} + t^{\text{fly}}_{g_k,i_{k\ell},i_{k,\ell+1}}, \qquad \ell < m_k. \tag{6.32}$$

对在第 ℓ 次访问时交付的货箱 b，其交付完成时刻为 $T_b = c_{k\ell}, \qquad b \in \mathcal{B}_{k\ell}$. 全部交付完成后，返航时刻为 $f_k = c_{km_k} + t^{\text{fly}}_{g_k,i_{km_k},0}$.

上述计算将整次交接结束作为该批货箱的完成时刻，未将到达服务区直接等同于完成交付。

6.2.2 硬时限与软期望时间

设 $\mathcal{B}_{\text{med}}$ 为医疗货箱集合，$\mathcal{B}_{\text{first}}$ 为首批保障货箱集合，d_b 为期望送达时刻，d^{first}_b 为首批截止时刻。两类硬时限分别表示为

$$T_b \le d_b, \qquad \forall b \in \mathcal{B}_{\text{med}}, \tag{6.33}$$

$$T_b \le d^{\text{first}}_b, \qquad \forall b \in \mathcal{B}_{\text{first}}. \tag{6.34}$$

若同一货箱同时具有两类标记，则两条约束同时成立。

其他物资的期望送达时间用于衡量及时性，不直接转化为硬截止时间。对于给定期望时间的货箱，定义延误 $\delta_b = \max\{0, T_b - d_b\}$, 并计算加权延误

$$D = \sum_{b:\, d_b \text{已给定}} \alpha_b \delta_b. \tag{6.35}$$

其中 α_b 为原始应急优先系数，故 D 的单位为优先系数加权秒。没有期望时间的货箱不参与该项计算，空白时间不能当作 0。硬时限始终作为可行性约束，不通过增加延误惩罚予以放宽。

6.3 无人机与共享电池资源约束

6.3.1 机型兼容与实体无人机占用

架次的实体无人机和电池必须与所选机型一致：$u_k \in \mathcal{U}_{g_k}, \qquad e_k \in \mathcal{E}_{g_k}$.

实体无人机从准备开始占用至返航落地，对应区间为 $\mathcal{I}^{\text{body}}_k = [s_k, f_k)$. 对于使用同一实体无人机的两个不同架次 k 和 k'，必须满足

$$u_k = u_{k'} \implies (f_k \le s_{k'} \ \vee \ f_{k'} \le s_k). \tag{6.36}$$

下一架次仍需完整计算固定准备及装载时间，不因连续执行而省略。

6.3.2 电池预留、充电与复用

电池从准备开始即被该架次预留。设架次 k 使用电池的返航 SOC 为 σ_k，其充电完成时刻为 $r_k^{\text{bat}} = f_k + \tau_{e_k}^{\text{chg}}(\sigma_k)$, 其中 $\tau_{e_k}^{\text{chg}}$ 采用题给两阶段充电模型。

该电池的完整占用区间为 $\mathcal{I}_k^{\text{battery}} = [s_k, r_k^{\text{bat}})$. 若两个架次使用同一电池，则

$$e_k = e_{k'} \implies \left(r_k^{\text{bat}} \le s_{k'} \ \vee \ r_{k'}^{\text{bat}} \le s_k\right). \tag{6.37}$$

该约束同时排除了电池任务重叠、任务与充电重叠，以及未充满就再次投入准备的情况。所有电池初始 SOC 为 100%，不同电池允许并行充电。机体和电池分别记录可用时刻，避免将返航落地误认为电池已经恢复可用。

6.4 多目标优化模型

在满足货箱、路线、容量、能源、硬时限和资源约束的前提下，定义运输任务完成时间 $C_{\max}$ 为最晚返航时刻，总能耗 E 为各架次能耗之和，架次数 N 为实际执行的运输架次数。其中 $\mathcal{K}$ 为实际执行的运输架次集合。

基准采用字典序目标

$$\text{lex min}\,(D, C_{\max}, E, N). \tag{6.38}$$

首先尽量减少加权延误，再比较全部运输任务的最晚返航时刻，随后比较总能耗和架次数。该排序是建模选择，不使用未经说明的权重将不同单位指标直接相加。

$C_{\max}$ 以最后一架运输无人机返回 O01 为准，不是最后一个货箱的交付时刻，也不包含最后一次返航后的补电时间。最后补电虽不延长 $C_{\max}$，仍在电池状态记录中保留。

由于货箱组批、访问顺序、离散资源选择和连续时间相互耦合，且能耗随载荷非线性变化，该问题具有混合离散与连续决策特征。本研究建立上述约束与目标后，采用启发式联合搜索获得可行调度，其结果不具备完整混合整数非线性模型的精确最优性证明。

6.5 多起点构造与模拟退火邻域搜索

6.5.1 可行方案构造与资源解码

利用多起点随机扰动构造完整可行方案，以各架次的货箱集合、机型和批次顺序作为搜索状态。对候选架次枚举所涉服务区的访问顺序，并检查质量、体积、逐段载荷、能耗和硬时限。同型号机体与电池分别维护可用时间，架次最早准备时刻为 $s_k^{\text{earliest}} = \max\{r_{u_k}^{\text{body}}, r_{e_k}^{\text{battery}}\}$. 机体返航后恢复可用，电池须充满后才能再次投入准备。候选方案变化时，后续资源安排与任务时刻均重新解码。

6.5.2 邻域与候选接受

模拟退火以一定概率接受较差候选，以扩大搜索范围[10]。批次合并和重组的邻域设计参考了带时限取送问题中的大邻域搜索思想[11]；本文采用的具体邻域与接受规则如下。

邻域操作包括机型变更、批次插入或交换、货箱移动或交换、批次合并和拆分，以及两个批次合并后重新分配货箱或按目的地重组。以完成时间和延误为主的辅助评价函数驱动模拟退火，允许暂时接受部分较差候选以离开局部解；最终保留方案仍按式 (6.38) 的字典序目标筛选。不满足硬约束的候选不进入接受比较，不能用惩罚项放宽医疗和首批截止时间。

6.5.3 搜索设置与范围

邻域搜索采用 20260925、20260926 和 20260927 三个随机种子，每组搜索时间预算为 180 s。每架次访问点数上限取 3 或 4，通过不同搜索设置构造候选方案。按照字典序目标筛选满足第二问约束的候选，并对选定方案独立核验。访问点数上限、邻域和壁钟时间均为计算限制，不是原题约束；未穷尽全部可行调度。

6.6 调度结果与分析

6.6.1 基准方案汇总

所得方案完成 80 箱按期交付。16 箱医疗物资与 30 箱首批保障物资均满足各自硬时限，两类标记允许重叠。结果见表 6.7。A、B、C 型分别执行 12、6、6 架次。各实体机均

表 6.7　第二问基准调度结果

指标	结果
交付箱数	80
加权延误 D	0
运输架次数	24
多服务区架次数	6
最晚返航时刻/s	6208.684865
总能耗/kWh	69.240573590
实际运输机/电池数量	8/14
最低返航 SOC	20.958111%
最小硬时限余量/s	194.866445

执行 3 架次，U08 的末次返航决定 $C_{\max}$。方案最晚返航时间为 103.478081 min，全部运输任务在给定库存内完成。与第一问的比较须同时考虑本问新增的资源、时限及多点访问约束。

表 6.8　所得方案的实体无人机任务分配

无人机	机型	架次数	累计占用/s	末次返航/s
U01	A	3	5390.093	5797.832
U02	A	3	5699.660	5817.252
U03	A	3	5215.296	5215.296
U04	A	3	5337.313	6113.782
U05	B	3	6024.529	6024.529
U06	B	3	5418.168	5418.168
U07	C	3	6195.854	6195.854
U08	C	3	6208.685	6208.685

图 6.4 展示 8 架实体无人机执行 24 个架次的时序。灰色区段表示准备与装载，彩色区段表示起飞至返航的占用，包含飞行和服务区交接；同一行的空白区段表示机体未被占用。U08 的最后一个架次决定运输任务完成时间。

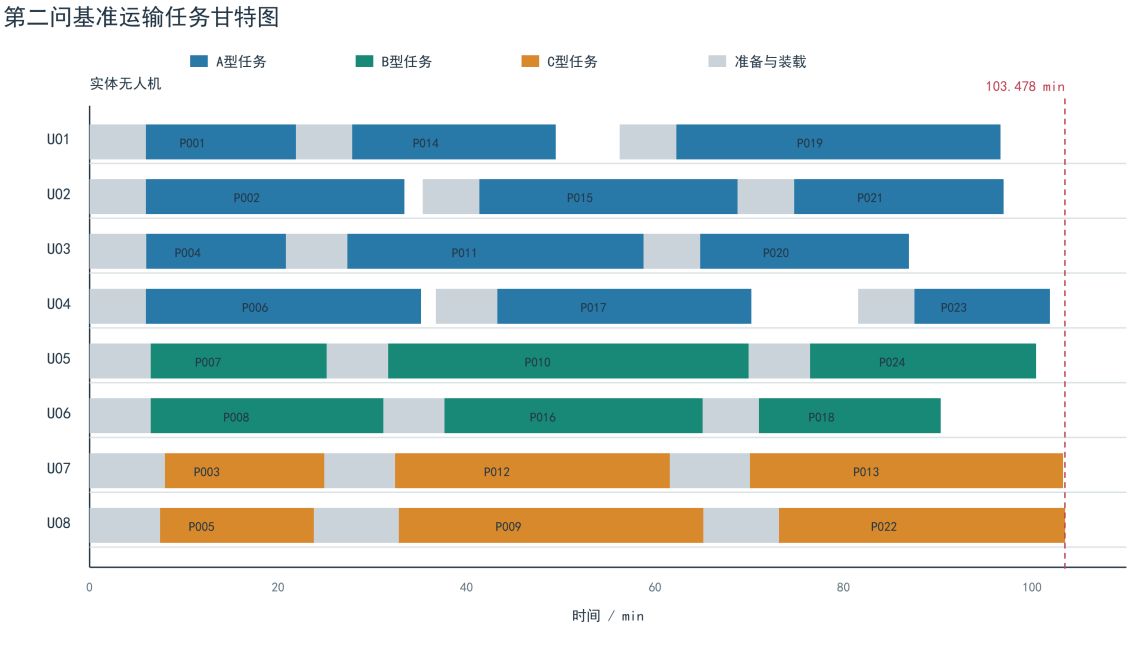


图 6.4　第二问运输无人机任务甘特图

6.6.2 多点架次的载荷与时间示例

选取 C 型无人机 U08 执行的 P009 架次，其使用电池 C-BAT-03，携带 8 箱物资，路线为 O01 → S006 → S007 → O01. 初始载荷 74 kg，在 S006 交付 4 箱后降至 37 kg，在 S007 交付其余 4 箱后空载返回。

表 6.9　P009 架次的逐航段载荷与能耗

航段	载荷/kg	起始/s	到达/s	能耗/kWh
O01→S006	74	1968.159	2301.527	2.011156
S006→S007	37	2625.527	3023.843	1.040271
S007→O01	0	3347.843	3907.628	1.438274

该架次准备开始与起飞时刻分别为 1428.158932 s 和 1968.158932 s，两站交付完成时刻为 2625.527260 s 和 3347.843322 s，均在到达之后计入完整交接时间。全架次能耗 4.489701013 kWh，返航 SOC 为 43.878737%。机体于 3907.628219 s 返航，但电池到 5956.922243 s 才充满，表明机体与能源恢复可用时间必须分别处理。

6.7 目标优先级对比与最优性边界

6.7.1 目标优先级比较

在本文搜索候选池内，分别按 $D \to C_{\max} \to E \to N$ 和 $C_{\max} \to D \to E \to N$ 筛选。以 10^{-7} s 作为完成时间的数值比较容差，两者选中相同方案。

两种排序得到相同结果仅反映所搜索范围，不证明及时性与完成时间在全部可行方案中没有冲突。

表 6.10　第二问候选池的目标优先级比较

排序	加权延误	完成时间/s	能耗/kWh	架次
$D \to C_{\max} \to E \to N$	0	6208.684865	69.240573590	24
$C_{\max} \to D \to E \to N$	0	6208.684865	69.240573590	24

6.7.2 最优性边界

由于 $D \geq 0$ 且当前方案达到 $D = 0$，第一层目标达到其下界。完成时间、能耗与架次数尚无全局最优证明；因此，本文将该方案报告为当前搜索得到、且通过独立验证的最好可行方案。

6.8 独立事件回放与可行性检查

独立程序重新读取原始货箱、设备和时限，采用 40 位十进制精度复算能耗与时刻，并按准备开始、返航充电、充满恢复可用三类事件进行回放，共 72 个运输资源事件。检查 80 箱无遗漏、无重复、无错投，质量、体积、逐段载荷、SOC 及全部硬时限均合规，机体占用和电池使用、充电、复用无冲突。最低返航 SOC 为 20.958111%，最小硬时限余量 194.866445 s，最大架次能耗重算差约 4.595×10^{-16} kWh。时间、能耗、质量、体积与 SOC 容差分别为 10^{-7} s、10^{-8} kWh、10^{-9} kg、10^{-12} m^3 及 10^{-10}。判定使用未舍入数据，同刻先释放后占用。全部 24 架次、80 箱交付和电池充电记录另附；第二问的运输验证不能替代第三问的通信检查。

7 问题三：通信约束下运输与中继联合调度

无人机辅助通信需要同时考虑空中节点位置、信道条件与能源约束[12]。相关旋翼无人机研究讨论了轨迹、通信任务与能耗之间的耦合[13]；本文在题给链路参数和中继功率规则下建立运输与中继联合模型。

问题三在第二问运输调度的基础上，引入运输无人机全过程通信保障要求。运输机在爬升、巡航、下降、交接及返航过程中，均需与固定网关建立可用链路，或通过中继无人机完成通信。由于链路状态随三维位置和地形遮挡变化，运输任务的路线、执行时刻与中继位置、服务时段之间存在耦合，因此需要联合安排运输与中继任务。

本问保留第二问的货箱、容量、能量、硬时限和资源约束，将第二问方案作为初始候选与对照，不预先固定其组批、路线、机型及任务开始时刻。优化过程中同时考虑配送及时性、联合完成时间、运输与中继总能耗及两类无人机总架次数。

7.1 三维轨迹与通信链路模型

7.1.1 运输无人机全过程轨迹

根据运输架次的实际执行时刻，将每一航段分解为垂直爬升、水平巡航和垂直下降，并在服务区加入交接停留阶段。对起止时刻分别为 t_a, t_b、起止位置分别为 $\boldsymbol{p}_a, \boldsymbol{p}_b$ 的运动阶段，采用分段线性轨迹

$$\boldsymbol{p}_k(t) = \boldsymbol{p}_a + \frac{t - t_a}{t_b - t_a} \left(\boldsymbol{p}_b - \boldsymbol{p}_a \right), \qquad t \in [t_a, t_b]. \tag{7.39}$$

对于交接停留阶段，$\boldsymbol{p}_a = \boldsymbol{p}_b$，运输机保持在服务区地面以上 30 m 的作业高度。

固定网关 G01 位于 O01，其通信端点位置为 $\boldsymbol{p}_G = (x_0, y_0, z_0 + h_G)$, 其中附件给定天线离地高度 $h_G = 20$ m。因此，网关通信端点不能直接放在 O01 地面。

全过程轨迹既用于判断服务区交接期间的通信，也用于检查各航段的爬升、巡航和下

降阶段。仅检查服务区位置或巡航轨迹，不能代表完整运输过程的通信状态。

7.1.2 双向链路预算

设通信端点 u 的发射功率为 P_u^{tx}，端点 u, v 的天线增益分别为 G_u, G_v，系统损耗为 L_{sys}。端点 v 的有效接收门限由接收灵敏度与衰落裕量组成：$P_v^{\mathrm{th}} = P_v^{\mathrm{sens}} + F_v$. 则方向 $u \to v$ 允许的最大传播损耗为

$$\mathcal{L}_{u\to v}^{\max} = P_u^{\mathrm{tx}} + G_u + G_v - L_{\mathrm{sys}} - P_v^{\mathrm{th}}. \tag{7.40}$$

双向通信要求两个方向均可用，因此取较严格的方向作为链路门限：

$$\mathcal{L}_{uv}^{\max} = \min\left\{\mathcal{L}_{u\to v}^{\max}, \mathcal{L}_{v\to u}^{\max}\right\}. \tag{7.41}$$

根据附件实际参数，载波频率为 2400 MHz，系统损耗为 3 dB，有效接收门限为 $-98+8 = -90$ dBm. 三类双向链路允许的最大损耗如表 7.11 所示。

表 7.11　双向通信链路损耗门限

链路类型	最大允许损耗/dB
运输机—固定网关	122
运输机—中继接入端	116
中继回传端—固定网关	126

中继接入端和回传端使用不同收发参数，不能将接入链路门限直接用于回传链路。

7.1.3 自由空间损耗与地形遮挡

自由空间传播损耗采用 ITU-R P.525-5 建议书的基本关系[14]，地形遮挡附加损耗按题目附件另行计入。

设端点 u, v 之间的三维距离为 $d_{uv}^{\mathrm{3D}}(t) = \|\boldsymbol{p}_u(t) - \boldsymbol{p}_v(t)\|_2$. 将距离换算为千米后，自由空间损耗为

$$\mathcal{L}_{uv}^{\mathrm{FS}}(t) = 32.45 + 20\log_{10} f_{\mathrm{MHz}} + 20\log_{10} d_{uv,\mathrm{km}}^{\mathrm{3D}}(t). \tag{7.42}$$

为判断地形遮挡，沿两通信端点之间的三维连线逐点检查 DEM 高程。若射线经过的地形高程达到或超过对应射线高度，则按所采用的 DEM 表面模型判定为存在遮挡。以 $\chi_{uv}(t)$ 表示该时刻两端点之间是否受到地形遮挡。

按附件规定，遮挡附加损耗为 10 dB，故总传播损耗和链路裕量分别为

$$\mathcal{L}_{uv}(t) = \mathcal{L}_{uv}^{\mathrm{FS}}(t) + 10\chi_{uv}(t), \tag{7.43}$$

$$M_{uv}(t) = \mathcal{L}_{uv}^{\max} - \mathcal{L}_{uv}(t). \tag{7.44}$$

当 $M_{uv}(t) \geq 0$ 时，该双向链路可用。

由此可见，存在遮挡并不一定断联，还需比较附加损耗后的总损耗与允许门限；同样，不存在遮挡也不一定满足链路要求，较大的传播距离仍可能导致损耗超限。

DEM 坐标按 PixelIsPoint 登记方式处理。节点表海拔与 DEM 高程差异保持不变，不通过抬高节点人为消除遮挡。在精细检查中，对射线穿越的像元以及擦边、过角位置采用保守地形检查。

7.2 中继任务、能耗与资源模型

7.2.1 悬停位置与转场剖面

设中继架次 r 的悬停位置为 $\boldsymbol{p}_r^{\mathrm{hov}} = (x_r, y_r, z_r)$. 悬停位置必须位于 DEM 覆盖范围内，且满足附件给出的离地高度上限：

$$0 < z_r - z_{\mathrm{DEM}}(x_r, y_r) \le 300\,\mathrm{m}. \tag{7.45}$$

中继从 O01 出发，到达悬停点后建链，服务结束后返回 O01。沿用前述工作假设，转场海拔取

$$H_r^{\mathrm{tr}} = \max\left\{ z_r^{\mathrm{DEM,max}} + 50,\ z_0,\ z_r \right\}. \tag{7.46}$$

其中 $z_r^{\mathrm{DEM,max}}$ 为 O01 与悬停点之间水平航迹对应的最高地形高程。

设 O01 至悬停点的水平距离为 d_r，中继巡航、爬升和下降速度分别为 v_R^{cr}、v_R^{+} 和 v_R^{-}，则去程和返程时间分别为

$$t_r^{\mathrm{out}} = \frac{H_r^{\mathrm{tr}} - z_0}{v_R^{+}} + \frac{d_r}{v_R^{\mathrm{cr}}} + \frac{H_r^{\mathrm{tr}} - z_r}{v_R^{-}}, \tag{7.47}$$

$$t_r^{\mathrm{back}} = \frac{H_r^{\mathrm{tr}} - z_r}{v_R^{+}} + \frac{d_r}{v_R^{\mathrm{cr}}} + \frac{H_r^{\mathrm{tr}} - z_0}{v_R^{-}}. \tag{7.48}$$

该剖面避免悬停海拔较高时出现负爬升或负下降。

7.2.2 有效服务时段

设中继架次 r 的准备开始时刻为 s_r，固定准备时间为 t_R^{prep}，建链时间为 t_R^{link}，则建链完成时刻为

$$c_r = s_r + t_R^{\mathrm{prep}} + t_r^{\mathrm{out}} + t_R^{\mathrm{link}}. \tag{7.49}$$

若服务结束时刻为 e_r，则中继返航时刻为 $f_r = e_r + t_r^{\mathrm{back}}$.

中继仅在 $[c_r, e_r]$ 内具有提供保障的资格，到达悬停点但尚未完成建链时不能提前服务。附件给定固定准备时间为 $180\,\mathrm{s}$，建链时间为 $30\,\mathrm{s}$，机体返航后周转时间为 $300\,\mathrm{s}$。

7.2.3 中继能耗分解

中继计划起飞总质量采用附件值 $m_R = 23.5\,\mathrm{kg}$。其巡航功率为 $1.15\,\mathrm{kW}$，悬停功率与通信附加功率分别为 $1.05\,\mathrm{kW}$ 和 $0.05\,\mathrm{kW}$。

水平往返能耗为

$$E_r^{\mathrm{hor}} = P_R^{\mathrm{cr}} \frac{2 d_r}{3600 v_R^{\mathrm{cr}}}. \tag{7.50}$$

爬升附加能耗为

$$E_r^{\mathrm{climb}} = \frac{m_R g_0 \left[(H_r^{\mathrm{tr}} - z_0) + (H_r^{\mathrm{tr}} - z_r) \right]}{\eta_R\, 3.6 \times 10^6}. \tag{7.51}$$

在建链阶段按悬停功率与通信附加功率之和计耗的工作假设下，

$$E_r^{\mathrm{link}} = (P_R^{\mathrm{hov}} + P_R^{\mathrm{com}}) \frac{t_R^{\mathrm{link}}}{3600}, \tag{7.52}$$

$$E_r^{\mathrm{service}} = (P_R^{\mathrm{hov}} + P_R^{\mathrm{com}}) \frac{e_r - c_r}{3600}. \tag{7.53}$$

全架次能耗为水平飞行、爬升、建链与悬停服务能耗之和。

中继能源组件可用能量为 $3.2\,\mathrm{kWh}$，返航 SOC 下限为 20%，因此 $E_r \le 0.8 \times 3.2 = 2.56\,\mathrm{kWh}$. 下降不另计附加能耗，准备、周转阶段也不加入缺乏参数依据的额外耗电。

7.2.4 机体与能源组件独立调度

附件提供 2 架中继无人机和 6 组共享能源组件，组件初始均为满电，再次使用前必须充满。中继机体从准备开始占用至返航并完成周转；能源组件从准备开始占用至返航后重新充满。同一机体或同一组件的占用区间不得重叠。

组件返航后立即开始充电，可与机体周转并行；下一架次只有在机体可用且所选组件已满电时才能开始准备。不同组件允许并行充电，不额外引入附件未给定的单充电器限制。

7.3 通信保障约束与联合目标

7.3.1 直连与中继保障选择

对运输架次 k 在飞行和交接期间的任意时刻 t，定义 $y_{k0}(t)$ 表示是否由 G01 直连保障，$y_{kr}(t)$ 表示是否由中继架次 r 保障。要求

$$y_{k0}(t) + \sum_{r} y_{kr}(t) = 1. \tag{7.54}$$

直连链路满足门限时优先使用直连。直连不可用时，只有同时满足下列条件的中继才能被选用：

$$y_{kr}(t) = 1 \quad \Longrightarrow \quad \begin{cases} c_r \le t \le e_r, \\ M_{kr}^{\text{access}}(t) \ge 0, \\ M_{rG}^{\text{backhaul}}(t) \ge 0. \end{cases} \tag{7.55}$$

若直连与所有有效中继均不能满足条件，则该时刻发生通信中断。

通信保障采用直连或单架中继的两段链路，不设置中继间多跳。附件未给出并发数量和切换耗时限制时，模型按链路条件允许一架中继同时服务多架运输机，并将保障切换耗时取为零。实际带宽与网络容量仍需实测核定。

上述约束描述连续时间下的通信要求，实际求解和验证采用离散检查及局部加密，其认证范围在后文单独说明。

7.3.2 联合优化目标

保留第二问的全部货箱交付、硬时限、容量、运输能量、机体与电池资源约束，同时加入中继飞行、能源和通信保障约束。

联合完成时间定义为

$$C_{\text{joint}} = \max\left\{\max_{k} f_k,\ \max_{r} f_r\right\}. \tag{7.56}$$

总能耗为运输与中继各架次能耗之和；总架次数为两类架次数之和。

基准采用字典序目标

$$\operatorname{lex\,min}\left(D, C_{\text{joint}}, E_{\text{total}}, N_{\text{total}}\right). \tag{7.57}$$

联合完成时间包含中继最终返航，但不包含最终补电和最终机体周转。医疗及首批保障硬时限始终保持，不能以延误罚项替代。

7.4 候选生成与联合搜索方法

7.4.1 空间候选与服务结构

采用水平网格、服务区位置和地形高点生成中继候选，离地高度取 150 米和 300 米，经 DEM 范围、高度及回传链路筛选后形成 597 个候选位置。时间候选组合涉及其中 49 个位置和 1904 组三架次结构。求解采用候选筛选与通信服务日历相结合的方法，检查中继服务窗口与运输轨迹的时间匹配。

7.4.2 通信服务日历参与运输调度

进一步采用一架中继持续服务、另一架先后执行两个中继任务的结构，将服务窗口、返航、机体周转、再次准备与建链时间纳入运输候选的可执行性检查。以运输可行候选作为搜索起点，调整运输机型、批次顺序、访问顺序及准备时刻，使需要中继的航段避开中继尚未建链或换场的时段。三处位置沿用候选 130、511 和 489，最终按实际通信需求重新确定服务起止时刻。运输时刻与中继服务窗口共同参与候选筛选。

7.4.3 计算限制

服务日历搜索种子为 20260928，时间预算 180 秒，包含最终细检查的实际耗时约 182.748 秒。候选运输批次最多涉及 4 个服务区；中继位置与三架次结构仍受限制。本研究服务日历搜索暂要求所有已给定期望时限均满足，以寻找 $D = 0$ 方案，这只是搜索限制，普通物资期望时间在原模型中仍为软目标。未穷举连续位置、任意组批和中继任务数量，也未证明全部离散候选范围内的最优性。

7.5 联合调度结果与权衡分析

7.5.1 中继位置与任务安排

所得方案使用 2 架中继机、3 组初始满电能源组件执行 3 个中继架次。悬停位置、任务时刻和能耗分别列于下表，所有位置均满足 DEM 范围及离地高度约束。

表 7.12　所得方案的中继悬停位置

架次	机体	经度/度	纬度/度	海拔/m
R001	R01	109.204167	23.048056	837.268860
R002	R02	109.270134	23.015271	627.188568
R003	R02	109.266111	23.056389	623.466492

表 7.13　所得方案的中继任务安排

架次	准备/s	建链完成/s	服务结束/s	返航/s	能耗/kWh
R001	325.265	1056.880	5721.485	6302.231	1.717471
R002	130.908	738.785	3329.431	3768.932	1.019583
R003	4315.907	5077.631	5506.123	6099.161	0.457479

R001、R002、R003 的返航 SOC 分别为 46.329029%、68.138039% 和 85.703779%。R002 于 3768.932416 s 返航，完成 300 秒周转后于 4068.932416 s 恢复机体可用，早于 R003 的准备时刻 4315.906989 s，两次机体占用不冲突。但 R002 的能源组件到 4683.137907 s 才充满，不能直接承接 R003。

图 7.5 展示运输轨迹上的直连判定及中继悬停位置。红色采样点对应相应飞行高度下直连不可用的位置，需结合中继服务时段判断保障状态；其分布并非地面静态覆盖范围。虚线连接中继与网关，用于标示回传关系。

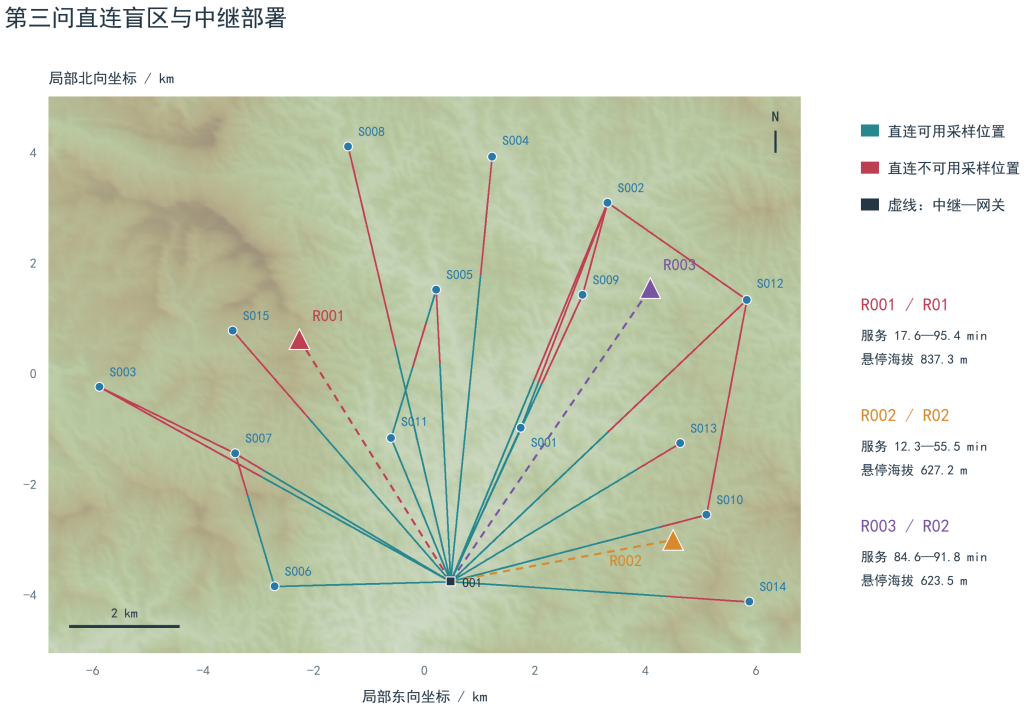


图 7.5　运输轨迹直连盲区与中继部署

7.5.2 联合方案汇总

80 箱全部按期交付，医疗及首批硬时限全部满足，$D = 0$。

表 7.14　第三问联合调度结果

指标	结果
运输/中继架次	24/3
运输最晚返航/s	6208.684865
联合最晚返航/s	6302.230771
运输能耗/kWh	69.240573590
中继能耗/kWh	3.194532893
总能耗/kWh	72.435106483
运输机/运输电池	8/14
中继机/能源组件	2/3

联合完成时间为 105.037180 min，由 R001 的最终返航决定，比运输最晚返航晚 93.545906 s。第二问与第三问选定的运输子方案具有相同的路线、机型和任务时刻。该运输子方案满足第二问的运输约束；第三问还需配置相应中继任务，并独立检查全过程通信，才能形成联合可行方案。

7.5.3 目标优先级比较

按 $D \to C_{\text{joint}} \to E_{\text{total}} \to N_{\text{total}}$ 与 $C_{\text{joint}} \to D \to E_{\text{total}} \to N_{\text{total}}$ 筛选当前候选池，均选中上述 $D = 0$、6302.230771 s、72.435106483 kWh 及 24 运输加 3 中继架次的方案。这不证明两个完整优化问题具有相同的全局最优解，尤其本研究服务日历搜索偏重全部按期交付的候选。

7.6 独立轨迹回放与通信验证

7.6.1 任务与资源检查

独立程序重读原始货箱、设备、通信参数和 DEM，重构运输全过程轨迹，复算中继飞行与能耗，检查所有机体、运输电池及中继组件的占用、周转和充电。两种优先级选中的方案均通过检查，运输最低返航 SOC 为 20.958111%，最小硬时限余量为 194.866445 s。

7.6.2 分辨率加密检查

检查覆盖爬升、巡航、下降、交接以及中继建链与服务边界，结果见表 7.15。

表 7.15　所得方案的通信分辨率核验

时间步长与空间检查	点数	失败点	最小裕量/dB
1 秒，5 米射线	36218	0	0.000703269
0.5 秒，2.5 米射线	72113	0	0.000356918
0.5 秒及局部加密，全穿越像元	73220	0	0.000155292

对端点裕量低于 0.02 dB 或保障源变化的区间细化至不超过 0.05 秒，对定位到的切换点二分至 10^{-7} s。最差所选链路裕量出现在 P010 巡航阶段 $t = 3860.254957$ s 附近。

图 7.6 以 P010 架次展示直连裕量、实际所选保障链路裕量及保障源切换。局部放大图取自独立回放记录，显示最小裕量附近各检查点的变化；连线用于辅助观察这些离散记录点。

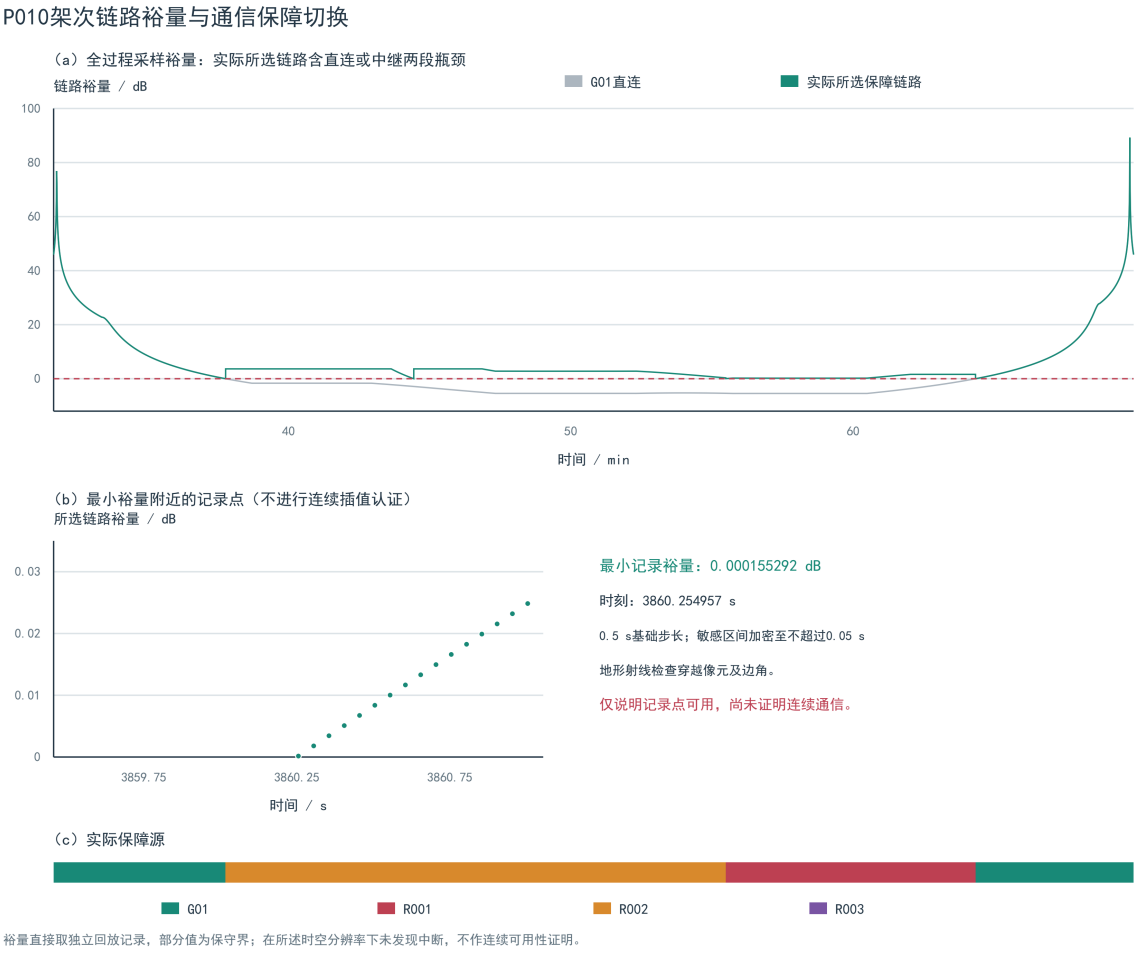


图 7.6　P010 架次的链路裕量与通信保障切换

7.6.3 验证结论与适用范围

数值验证表明：**在所述时空分辨率下未发现通信中断**。离散检查仍无法严格排除采样间隙中的短暂失效，因而连续通信可用性尚未得到数学证明；全部 196 个运输阶段保守列为尚未严格认证区间。248 条通信保障记录及其任务关联用于第四问冻结。加权延误 $D = 0$ 达到第一层目标的非负下界，后续目标尚无全局最优证明。标准航程能量标定、爬升效率解释、未计运输交接悬停能耗，以及中继多机并发和零额外切换耗时均为模型假设，实际应用需以设备试验和通信容量测量校准。

8 问题四：救援任务分区与资源配置优化

8.1 问题分析与建模思路

问题四以第三问调度方案为固定输入，将 15 个服务区分别划分为 $K = 2$ 和 $K = 3$ 个任务组，确定各组独立执行运输与通信保障任务所需的资源。分区决策不调整货箱组批、访问顺序及任务时刻；研究重点是集中共享的资源转为分组独立使用后，配置规模与库存缺口如何变化。

本问的主要困难在于：运输架次可能同时访问多个服务区，使服务区之间形成不可拆分的绑定关系；中继架次可能为多个任务组提供保障，需要明确其独立执行方式；电池和能源组件的复用还受到充电时间约束，不能直接将第三问使用的不同编号数量视为最小配

置。

为此，本文首先构造必须同组的服务区块，再通过固定时序下的区间重叠计算各组最小资源需求，最后完整枚举合法分区，按资源缺口、配置规模和工作量均衡性进行字典序优化。

8.2 冻结方案与解释口径

8.2.1 冻结调度的定义

以第三问基准方案作为冻结输入，保持以下内容不变：运输货箱及其目的地、架次访问顺序、运输机型、准备与起飞时刻、交付完成与返航时刻，以及中继位置、建链与服务时段、返航时刻和实际通信保障关系。

冻结并不要求所有资源承接编号保持不变。本文基准允许组内同型号资源重新编号，但不得因此改变任务内容、物理轨迹或执行时序。

设冻结运输架次集合为 $\mathcal{T}$，原中继架次集合为 $\mathcal{R}$，服务区集合为 $\mathcal{V} = \{\mathrm{S001}, \ldots, \mathrm{S015}\}$. 分区后，每个服务区恰好属于一个非空任务组，各组在任务执行期间不得相互借用机体、电池或能源组件。

8.2.2 跨组中继与资源编号假设

由于题目未明确跨组中继的独立执行方式以及资源编号是否必须保留，本文采用以下**分区建模假设**作为基准解释：

1. 允许组内同型号机体、电池及能源组件重新编号和合法复用，仅改变资源承接关系，不改变任何任务时刻。
2. 当一个冻结中继架次实际保障多个任务组时，各相关组分别配置该架次的完整任务副本。每个副本保留原准备、转场、建链、服务、返航和周转过程，并独立计入资源占用与能耗。
3. 同一组内对同一原中继架次的多次引用只计一个副本，不同组的副本由独立资源执行。
4. 能源组件返航后立即充电，充电可与中继机体周转并行；不增加附件未规定的充电器数量或准备工位限制。

本文同时比较禁止复制中继架次、继承原实体编号两种替代解释。不额外施加地理连通、服务区数量相等或空间紧凑等约束。

8.3 必须同组关系与分区变量

设 $\mathcal{V}_k$ 为运输架次 k 访问的服务区集合。由于同一运输架次不能拆分到不同任务组，对任意 $i, j \in \mathcal{V}_k$ 均有 $\mathrm{group}(i) = \mathrm{group}(j)$.

将共同出现在一个运输架次中的服务区连接，通过并查集求取传递闭包。实际计算得到 8 个不可拆分服务区块：

$$\mathcal{B}_1 = \{\mathrm{S001}, \mathrm{S002}, \mathrm{S009}, \mathrm{S010}, \mathrm{S012}\}, \qquad \mathcal{B}_2 = \{\mathrm{S003}, \mathrm{S006}, \mathrm{S007}\},$$
$$\mathcal{B}_3 = \{\mathrm{S004}\}, \qquad \mathcal{B}_4 = \{\mathrm{S005}, \mathrm{S011}\},$$
$$\mathcal{B}_5 = \{\mathrm{S008}\}, \qquad \mathcal{B}_6 = \{\mathrm{S013}\},$$
$$\mathcal{B}_7 = \{\mathrm{S014}\}, \qquad \mathcal{B}_8 = \{\mathrm{S015}\}.$$

这些块由第三问运输架次的共同访问关系求传递闭包得到，分区决策作用于上述不可拆分块。

记块集合为 $\mathcal{B}$，定义二元变量 $z_{a\ell}$ 表示服务区块 a 是否划入任务组 ℓ。分区约束为

$$\sum_{\ell=1}^{K} z_{a\ell} = 1, \qquad a \in \mathcal{B}, \tag{8.58}$$

$$\sum_{a\in\mathcal{B}} z_{a\ell} \geq 1, \qquad \ell = 1, \ldots, K. \tag{8.59}$$

由于同一运输架次访问的全部服务区已合并至同一块，上述约束能够保证运输任务完整归属。

设 $a(k)$ 为运输架次 k 所属的服务区块，$\mathcal{H}_r$ 为原中继架次 r 实际保障的运输架次集合。定义 $c_{r\ell}$ 表示任务组 ℓ 是否需要中继 r 的完整副本，则

$$c_{r\ell} \geq z_{a(k)\ell}, \qquad k \in \mathcal{H}_r, \tag{8.60}$$

$$c_{r\ell} \leq \sum_{k\in\mathcal{H}_r} z_{a(k)\ell}, \qquad c_{r\ell} \in \{0, 1\}. \tag{8.61}$$

其中，$\mathcal{H}_r$ 由第三问实际通信保障区间确定，不以地理上可能覆盖的服务区替代实际保障关系。

8.4 固定时序下的最小资源配置

8.4.1 资源占用区间

本文考虑八类资源，依次为 A、B、C 型运输机，A、B、C 型电池，中继机及中继能源组件。下标中的 D 表示机体，B 表示电池或能源组件，R 表示中继机型。对应的资源需求向量为

$$\boldsymbol{R} = (R_{\mathrm{A,D}}, R_{\mathrm{B,D}}, R_{\mathrm{C,D}}, R_{\mathrm{A,B}}, R_{\mathrm{B,B}}, R_{\mathrm{C,B}}, R_{\mathrm{R,D}}, R_{\mathrm{R,B}}). \tag{8.62}$$

对应库存为 $\boldsymbol{I} = (4, 2, 2, 6, 4, 4, 2, 6)$.

设运输架次 k 的准备开始和返航时刻分别为 s_k 和 f_k，返航 SOC 为 σ_k。设中继架次 r 的相应时刻为 s_r、f_r，返航 SOC 为 σ_r，机体周转时间为 $\tau^{\mathrm{turn}} = 300\,\mathrm{s}$。运输机体占用至返航，电池占用至充满；中继机体占用至返航后完成周转，能源组件占用至充满。充电时间沿用第四节的两阶段模型。全部区间采用半开形式，同一时刻先释放资源、后开始新的占用。能源区间延伸至恢复满电，确保复用前满足满电要求。最后一次返航后的充电不计入联合任务完成时间，但保留在能源恢复记录中。

8.4.2 峰值需求与具体资源分配

对同类型、可互换且仅受占用区间约束的资源，采用区间划分中的贪心分配方法[15]，构造达到最大重叠数下界的具体编号方案。

记 $\mathcal{J}_{\ell\gamma}$ 为任务组 ℓ 中占用资源类型 γ 的任务集合，任务 j 的占用区间为 $[a_j, b_j)$。该组该类型资源的最小配置为

$$n_{\ell\gamma} = \max_{t} \sum_{j\in\mathcal{J}_{\ell\gamma}} \mathbf{1}\{a_j \leq t < b_j\}. \tag{8.63}$$

式 (8.63) 具有下界与可实现性两方面依据。任意时刻同时占用的任务必须由不同实体承担，因此最大重叠数构成资源需求下界。另一方面，将任务按开始时刻排序，优先复用已经释放的资源，仅在所有已有资源均被占用时增加编号，即可构造恰好达到最大重叠数的资源分配。

该结论成立的前提是同类型资源可互换、所有任务从 O01 出发并返回 O01，且不存在额外的兼容性或转场约束。不同机型电池分别计算，不允许混用。

未分区且资源可共享时，重新计算得到最小配置 $\boldsymbol{R}^0 = (4,2,2,6,4,4,2,3)$, 向量顺序为 A、B、C 型运输机，A、B、C 型电池，中继机和中继能源组件。在 4315.906989 s，R003 开始预留能源，R001 仍在使用其组件，R002 组件尚在充电，三组能源占用同时存在，因此至少需要 3 组中继组件。R002 组件到 4683.137907 s 才充满，晚于 R003 准备开始，不能复用承接。因此，该冻结时序下的中继能源组件最小需求为 3 组，与具体可用时刻的核算结果一致。

8.5 分区优化目标

8.5.1 需求、缺口与库存剩余

对资源类型 γ，定义总需求、缺口及库存剩余为

$$R_\gamma = \sum_{\ell=1}^{K} n_{\ell\gamma}, \tag{8.64}$$

$$\Delta_\gamma = \max\{0, R_\gamma - I_\gamma\}, \tag{8.65}$$

$$U_\gamma = \max\{0, I_\gamma - R_\gamma\}. \tag{8.66}$$

各类型缺口分别报告，不允许用某类资源的富余抵消另一类短缺。由于本问需要分析资源缺口，库存不足的分区仍参与比较，但须明确其执行依赖于补充相应资源。

无采购价格数据时，采用归一化缺口与配置规模：

$$G = \sum_{\gamma} \frac{\Delta_\gamma}{I_\gamma}, \tag{8.67}$$

$$S = \sum_{\gamma} \frac{R_\gamma}{I_\gamma}. \tag{8.68}$$

两者均为无量纲比较指标，表示八类资源等权归一化后的缺口和规模，不解释为实际经济成本。

8.5.2 工作量与字典序目标

定义任务组 ℓ 的工作量为机体占用时间总和：

$$W_\ell = \sum_{k\in\mathcal{T}_\ell} (f_k - s_k) + \sum_{r\in\mathcal{R}} c_{r\ell}(f_r + \tau^{\text{turn}} - s_r). \tag{8.69}$$

该指标包括中继周转，不包括电池充电。

以工作量变异系数衡量组间不均衡：

$$\mathrm{CV} = \frac{\sqrt{\frac{1}{K}\sum_{\ell=1}^{K}(W_\ell - \overline{W})^2}}{\overline{W}}, \qquad \overline{W} = \frac{1}{K}\sum_{\ell=1}^{K} W_\ell. \tag{8.70}$$

基准采用字典序目标

$$\mathrm{lexmin}(G, S, \mathrm{CV}). \tag{8.71}$$

即先减少库存缺口，再控制配置规模，最后在前两层目标最优的方案中改善工作量均衡。该优先级属于建模选择。

8.6 求解方法与最优性范围

对 8 个不可拆分块进行无标签非空分区枚举，两组和三组的合法分区分别为 127 种和 966 种。预先计算 $2^8-1=255$ 个非空子集的资源需求与工作量，再评价各分区目标。比较时对 $12G$ 和 $12S$ 使用整数值，均衡层比较 CV^2，并列容差为 10^{-12}。全部合法分区均已检查，两个基准分区在该容差下均为唯一最优。该最优性结论适用于冻结的第三问基准方案、完整中继复制、组内重新编号及本文采用的目标定义；运输、通信和分区共同决策的全局最优性尚未证明。

8.7 分区与配置结果

8.7.1 两组分区

记 $\mathcal{V}$ 为全部 15 个服务区。两组方案将 S008 单列为第二组，其余 14 个服务区归入第一组；各组资源需求见表 8.16。由表 8.16 可见，两组方案缺少 1 架 C 型运输机和 1 架中继机；中继能源组件剩余 2 组。目标为 $G=1$、$S=8.666667$、$\mathrm{CV}=0.735372$。

8.7.2 三组分区

三组方案将 S008 与 S015 各单列一组，其余 13 个服务区构成第一组。各组资源需求同见表 8.16。三组方案缺少 1 架 B 型运输机、1 架 C 型运输机、1 组 B 型电池和 2 架中继机；组件剩余 1 组，详细需求见表 8.16。目标为 $G=2.25$、$S=10.083333$、$\mathrm{CV}=0.914171$。

表 8.16　两种基准分区的资源需求与缺口

类型	库存	两组需求	两组缺口	三组需求	三组缺口
A 型运输机	4	4	0	4	0
B 型运输机	2	2	0	3	1
C 型运输机	2	3	1	3	1
A 型电池	6	6	0	6	0
B 型电池	4	4	0	5	1
C 型电池	4	4	0	4	0
中继机	2	3	1	4	2
中继组件	6	4	0	5	0

表 8.17　分区的各组工作量

K	组	箱数	质量/kg	体积/m³	运输架次	中继任务	工作量/s
2	G1	75	713	1.882	23	3	55588.142
2	G2	5	45	0.129	1	1	8476.668
3	G1	72	688	1.815	22	3	53759.898
3	G2	5	45	0.129	1	1	8476.668
3	G3	3	25	0.067	1	1	8105.209

图 8.7 在相同坐标范围内对比两组与三组方案。节点颜色表示任务组归属，紫色虚线以 S005 与 S011 为例标示运输任务形成的必须同组关系；图中不绘制地理分区边界，也不额外要求各组空间连通。

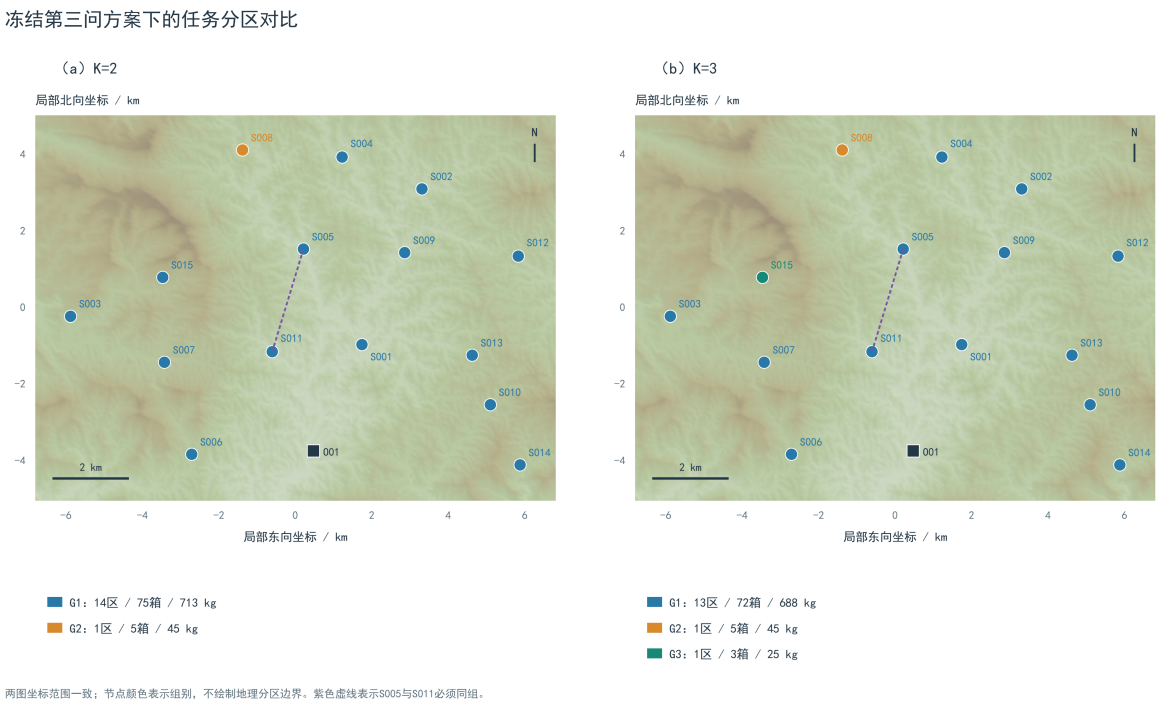


图 8.7　两组与三组任务分区对比

8.8 资源缺口与额外配置的来源

8.8.1 组间错峰复用损失

两组中，大组在 $t = 0$ 由 P003 与 P005 同时占用两架 C 型机，S008 组的 P012 另需一架，组间禁止调配导致总需求 3 架。大组由 R001、R002 重叠占用两架中继机，S008 组另需 R001 完整副本，因此中继需求也是 3 架。三组再独立配置 S015 组，需要额外保有 B 型机、B 型电池及中继机。各组峰值不必同刻出现；独立执行要求求各组峰值之和，而不是分组后的全局峰值。

图 8.8 对比未分区最小配置、两组及三组需求。黑色短线表示各类库存，红色柱段表示对应资源缺口；中继能源组件虽然因分区增加配置，但仍未超过库存。不同型号及类型的资源应分别核算，不能以其他类别的富余抵消缺口。

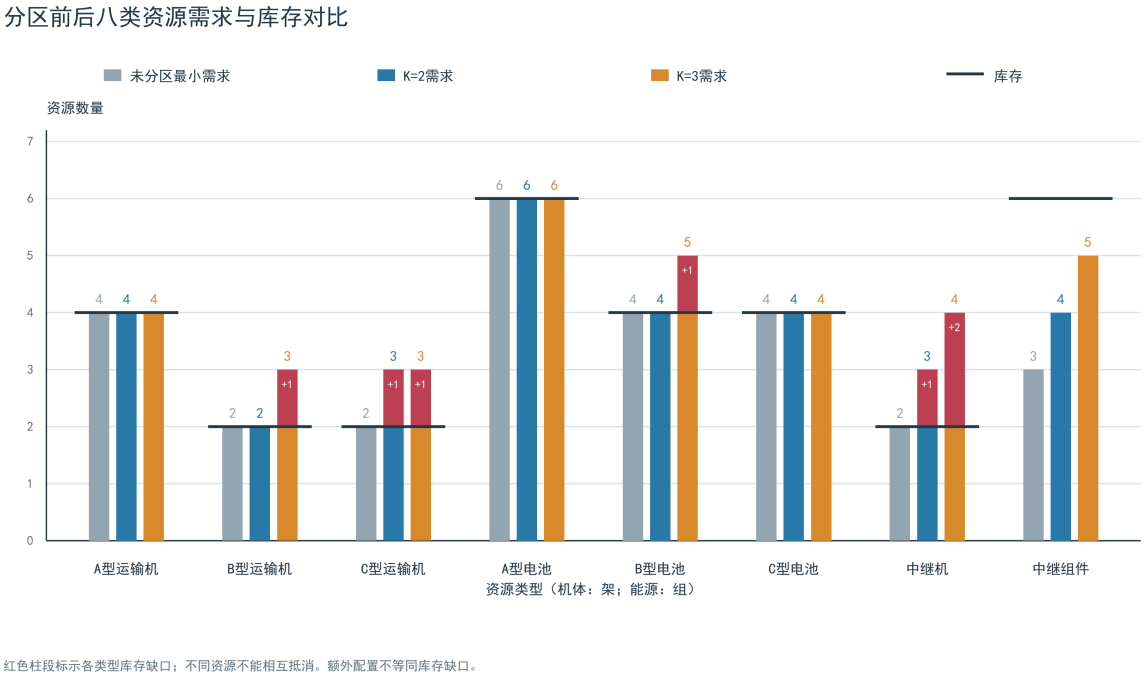


图 8.8　分区前后各类资源需求与库存对比

8.8.2 库存剩余、额外配置与任务重复

库存剩余为 $U_\gamma = \max(0, I_\gamma - R_\gamma)$，额外配置为 $A_\gamma = R_\gamma - R_\gamma^0$，完整中继复制引起的新增任务与能耗另行统计，三者不能混称冗余。两组相较未分区增加 1 架 C 型机、1 架中继机及 1 组组件；三组增加 1 架 B 型机、1 架 C 型机、1 组 B 型电池、2 架中继机和 2 组组件。增加组件配置不等于组件库存短缺。

两组对 R001 额外复制一次，三组对 R001 额外复制两次，故中继任务总数分别为 4 次与 5 次。两组和三组的复制能耗分别为 1.717471065 kWh 和 3.434942131 kWh。对应新增机体占用分别为 6276.966104 s 和 12553.932208 s，新增能源组件占用分别为 7174.688724 s 和 14349.377449 s。完整复制保持物理任务时刻，故两种分区仍有 $D = 0$、$C_{\text{joint}} = 6302.230771$ s。两组与三组的运输、中继总能耗分别为 74.152577548 kWh 和 75.870048613 kWh。

8.9 目标优先级与解释口径比较

8.9.1 配置规模与工作量均衡

将 $G \to S \to \text{CV}$ 改为 $G \to \text{CV} \to S$，两种 K 均选中相同基准分区。若将均衡置于首位，则可降低工作量差异，但资源缺口明显增加。

8.9.2 禁止复制中继架次

在禁止复制中继架次的解释下，同一中继实际保障的运输任务必须归入同组。将这些绑定加入运输关系并求传递闭包后，15 个服务区形成一个不可拆分集合：R001 和 R002 共同关联含 S001、S002、S009、S010、S012 的运输块，R003 关联的 S004、S005 也已与 R001 连通，最终覆盖全部服务区。因此，在禁止复制这一解释下，$K = 2$ 和 $K = 3$ 均不存在合法非空分区。

表 8.18　冻结方案的目标优先级比较

K	排序	G	S	CV
2	$G \to S \to \mathrm{CV}$	1.000000	8.666667	0.735372
2	$G \to \mathrm{CV} \to S$	1.000000	8.666667	0.735372
2	$\mathrm{CV} \to G \to S$	4.500000	12.500000	0.005234
3	$G \to S \to \mathrm{CV}$	2.250000	10.083333	0.914171
3	$G \to \mathrm{CV} \to S$	2.250000	10.083333	0.914171
3	$\mathrm{CV} \to G \to S$	7.333333	15.333333	0.107693

8.9.3 继承原实体编号

按各组使用的不同原编号分别计数；跨组出现的同名编号须各组独立配置。在固定基准分区下，继承编号所得配置与组内重新编号的最小资源向量一致；按继承编号口径重新优化分区，也得到相同的基准需求。

8.10 独立事件回放与结果有效性

独立程序重读原始参数，复算货箱交付、能耗、SOC、充电与周转，并回放组内资源事件。15 个服务区无遗漏或重复，同架次访问点同组；80 箱及其交付时刻保持不变，16 箱医疗物资和 30 箱首批保障物资均满足硬时限，最小余量为 194.866445 s。248 条实际通信保障区间得到继承，中继副本完整，组内资源无冲突且不跨组共享。共 12 个导出场景通过检查；独立枚举还核对了 127 个两组分区和 966 个三组分区，得到 10 项目标核对证书。冻结输入 SHA-256 在检查前后保持一致。时间、能耗、SOC、质量和体积容差分别为 10^{-7} s、10^{-8} kWh、10^{-10}、10^{-9} kg 和 10^{-12} m^3。检查确认所列配置可以承接冻结任务，但两种 K 均存在库存缺口。通信结论沿用第三问的离散验证范围；相同中继副本的共址干扰和实际网络容量尚未另行认证。

8.11 结果分析

冻结方案下，两组的资源缺口、配置规模、工作量变异系数和复制能耗均低于三组，因此按既定字典序目标优先采用两组。分区资源代价主要来自独立保有设备后失去跨组错峰复用机会，以及同一中继任务被多个组使用时所需的完整任务副本。上述结论以第三问的任务时序和实际通信保障关系为固定输入，依赖完整复制、组内同型号资源可重新编号等工作假设。在此口径下，现有库存均不能独立承接两种分区，需要按资源类型补足缺口。

9 模型评价

模型的主要特点是四问共用地形、载荷、能耗和资源占用规则，并对求解结果进行独立回放，减少逐段载荷、满电复用和跨问数据继承中的口径差异。批次重组、机型选择和通信服务日历共同参与调度：第二问以 24 个运输架次完成 80 箱按期交付，第三问配置 3 个中继架次，在所述时空分辨率下未发现通信中断。

第二、三问尚未穷尽货箱分配、运输顺序和连续中继位置。零延误证明第一层目标达到下界，但完整字典序目标尚无全局最优证明。通信检查属于有限分辨率的数值验证，仍可能遗漏采样间隙中的短暂失效。能量标定、爬升效率、交接悬停能耗简化及无线并发容量等假设，还需通过实测能耗和通信容量数据检验。

此外，分区要求资源不跨组调配，因而会产生额外配置及中继任务复制能耗。第四问的最优性仅针对冻结时序、完整中继复制和组内重新编号口径，不能扩展为运输、通信与分区共同决策时的全局最优性。

参考文献

[1] 新华社. 记者手记：抵近广西横州镇龙乡[EB/OL]. （2026-07-09）[2026-09-26]. https://www.xinhuanet.com/politics/20260709/acf8e4b353304bb78007d8224f1cd2ef/c.html.

[2] 新华每日电讯. 三进“孤岛乡”[EB/OL]. （2026-07-12）[2026-09-26]. https://www.news.cn/local/20260712/8bd1f64af3124569b5cf4415fc013acd/c.html.

[3] DJI Enterprise. Matrice 350 RTK Specifications[EB/OL]. [2026-09-26]. https://enterprise.dji.com/matrice-350-rtk/specs.

[4] Doodle Labs. Sense：Interference Avoidance[EB/OL]. [2026-09-26]. https://doodlelabs.com/blog/sense-interference-avoidance-release.

[5] Copernicus Data Space Ecosystem. Copernicus DEM：Global and European Digital Elevation Model[DB/OL]. [2026-09-26]. https://dataspace.copernicus.eu/explore-data/data-collections/copernicus-contributing-missions/collections-description/COP-DEM. DOI:10.5270/ESA-c5d3d65.

[6] DORLING K，HEINRICHS J，MESSIER G G，et al. Vehicle Routing Problems for Drone Delivery[J/OL]. IEEE Transactions on Systems，Man，and Cybernetics：Systems，2017，47（1）：70-85. DOI:10.1109/TSMC.2016.2582745.

[7] ZHANG J，CAMPBELL J F，SWEENEY D C，et al. Energy Consumption Models for Delivery Drones：A Comparison and Assessment[J/OL]. Transportation Research Part D：Transport and Environment，2021，90：102668. DOI:10.1016/j.trd.2020.102668.

[8] Texas Instruments. Li-Ion Battery Charger Solution Using an MSP430 MCU：SLAA287B[EB/OL]. （2022）[2026-09-26]. https://www.ti.com/lit/an/slaa287b/slaa287b.pdf.

[9] BELLMAN R. Dynamic Programming[M]. Princeton：Princeton University Press，1957.

[10] KIRKPATRICK S，GELATT C D，Jr，VECCHI M P. Optimization by Simulated Annealing[J/OL]. Science，1983，220（4598）：671-680. DOI:10.1126/science.220.4598.671.

[11] ROPKE S，PISINGER D. An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows[J/OL]. Transportation Science，2006，40（4）：455-472. DOI:10.1287/trsc.1050.0135.

[12] ZENG Y，ZHANG R，LIM T J. Wireless Communications with Unmanned Aerial Vehicles：Opportunities and Challenges[J/OL]. IEEE Communications Magazine，2016，54（5）：36-42. DOI:10.1109/MCOM.2016.7470933.

[13] ZENG Y，XU J，ZHANG R. Energy Minimization for Wireless Communication With Rotary-Wing UAV[J/OL]. IEEE Transactions on Wireless Communications，2019，18（4）：2329-2345. DOI:10.1109/TWC.2019.2902559.

[14] International Telecommunication Union. Recommendation ITU-R P.525-5：Calculation of Free-Space Attenuation[S/OL]. （2024）[2026-09-26]. https://www.itu.int/rec/R-REC-P.525-5-202411-I/en.

[15] KLEINBERG J，TARDOS É. Algorithm Design[M]. Pearson/Addison-Wesley，2006.

附录 A　人工智能产品辅助答题规范说明

本文在建模、程序实现、结果核验与文字整理过程中使用了人工智能产品辅助。人工智能提供的建议以题目附件、实际运行结果和独立检查记录为依据进行核对；模型设定、结果解释及论文内容由参赛队负责。

人工智能产品使用情况	
工具名称：DeepSeek Harness **开发机构：**幻方量化 **模型版本：** Deepseek-V4.1-Flash	（1）辅助梳理四个问题的建模思路、变量、约束及计算口径。 （2）辅助编写和调试 Python 求解、制图及独立核验程序；数值结果取自程序实际运行。 （3）辅助整理参考资料、LaTeX 结构与学术表述。
算法与技术组合的关键开发框架	
程序运行环境为 Windows 和 Python 3.12。模型求解使用动态规划、模拟退火、并查集、区间扫描与区间着色等方法；对应算法实现及复算入口见附录 B 和随附代码。	
数据分析与处理工具	
`NumPy` 用于数值计算，`openpyxl` 用于读取和写入表格，`Pillow` 用于图像处理与结果制图。原始附件、预处理数据与独立核验记录构成数值结论的复算依据。	

附录 B　关键代码展示

以下展示四问求解与核验所用程序的关键函数节选。代码取自与正文结果对应的工程版本；外部对象、数据读取及辅助函数由完整工程提供。各片段用于展示关键计算过程，完整源码与复算方式见随附代码来源清单。

问题一：单点往返能耗与字典序动态规划关键代码

```
def energy(site, g, q):
    m = models[g]
    go = legs['O01', site]
    back = legs[site, 'O01']
    L = m['L0'] - (m['L0'] - m['LF']) * (q / m['Q']) ** 1.5
    eh = m['Eu'] * float(go['horizontal_distance_m']) / L
    eu = (m['m0'] + q) * 9.80665 * float(go['climb_m']) / (m['eta'] * 3600000.0)
    rh = m['Eu'] * float(back['horizontal_distance_m']) / m['L0']
    ru = m['m0'] * 9.80665 * float(back['climb_m']) / (m['eta'] * 3600000.0)
    return eh + eu + rh + ru

def solve_site(s, rho, order):
    pp = [p for p in patterns[s] if p['soc'] >= rho]

    @lru_cache(None)
    def dp(rem):
        if not any(rem):
            return ((0, Decimal(0), Decimal(0)), ())
        anchor = next((k for k, n in enumerate(rem) if n))
        best = None
        for ix, p in enumerate(pp):
            v = p['vector']
            if not v[anchor] or any((a > b for a, b in zip(v, rem))):
                continue
            sub = dp(tuple((b - a for a, b in zip(v, rem))))
            if sub is None:
                continue
            c = sub[0]
            cost = (c[0] + 1, c[1] + Decimal(str(p['E'])), c[2] + Decimal(str(p[
 'T'])))
            key = tuple((cost[k] for k in order))
            if best is None or key < best[0]:
                best = (key, cost, (ix,) + sub[1])
        return None if best is None else (best[1], best[2])
    ans = dp(states[s])
    missing = [bid for k, idlist in enumerate(groups[s][1]) for bid in idlist if
 not any((p['vector'][k] > 0 for p in pp))]
    return (None if ans is None else [pp[i] for i in ans[1]], dp.cache_info().
 currsize, len(pp), missing)
```

问题二：两阶段充电与共享电池调度关键代码

```
def charge(self, g, soc):
    full = self.batts[g][0][1]
    return full * (0.65 * (0.9 - soc) / 0.9 + 0.35) if soc < 0.9 else full * 0.35
  * (1 - soc) / 0.1
```

```
def decode(state, full=False):
    ur = {g: [(0, u) for u in pr.units if pr.units[u] == g] for g in 'ABC'}
    br = {g: [(0, b) for b, _ in pr.batts[g]] for g in 'ABC'}
    D = E = C = 0
    plan = []
    for g, ids in state:
```

```
        ut, u = min(ur[g])
        bt, b = min(br[g])
        start = max(ut, bt)
        opts = []
        for dur, en, chg, hard, due, route in profiles(g, ids):
            if start > hard + 1e-08:
                continue
            delay = sum((w * max(0, start + off) for off, w in due))
            opts.append((delay, dur, en, chg, route))
        if not opts:
            return None
        delay, dur, en, chg, route = min(opts)
        finish = start + dur
        D += delay
        E += en
        C = max(C, finish)
        ur[g].remove((ut, u))
        ur[g].append((finish, u))
        br[g].remove((bt, b))
        br[g].append((finish + chg, b))
        if full:
            p = pr.evaluate(g, ids, route, start)
            assert p is not None
            p.update(unit=u, battery=b, sortie=f'P{len(plan) + 1:03d}',
  charge_start=p['finish'], battery_ready=p['finish'] + chg)
            plan.append(p)
    return plan if full else (D, C, E, len(state))
```

问题三：地形链路裕量计算关键代码

```
def margin(self, a, b, typ, method='cells', spacing=5):
    distance = float(np.linalg.norm(a - b))
    free = 32.45 + 20 * math.log10(self.f) + 20 * math.log10(max(distance /
  1000, 1e-09))
    limit = self.threshold[typ]
    if free + self.obs <= limit:
        return (limit - free - self.obs, True)
    if free > limit:
        return (limit - free, False)
    if method == 'sample':
        n = max(1, math.ceil(float(np.linalg.norm((a - b)[:2])) / spacing))
        ts = np.linspace(0, 1, n + 1)
        x0, y0 = self.grid(a)
        x1, y1 = self.grid(b)
        cs = np.floor(x0 + (x1 - x0) * ts).astype(int)
        rs = np.floor(y0 + (y1 - y0) * ts).astype(int)
        blocked = bool(np.any(self.dem[rs, cs] >= a[2] + (b[2] - a[2]) * ts - 1e
  -09))
    else:
        t, rs, cs = self.ray_cells(a, b)
        z = a[2] + (b[2] - a[2]) * t
        blocked = bool(np.any(self.dem[rs, cs] >= np.minimum(z[:-1], z[1:]) - 1e
  -09))
        x0, y0 = self.grid(a)
        x1, y1 = self.grid(b)
        xx = x0 + (x1 - x0) * t
        yy = y0 + (y1 - y0) * t
        for dc, dr in [(1e-08, 1e-08), (-1e-08, 1e-08), (1e-08, -1e-08), (-1e
  -08, -1e-08)]:
            cc = np.floor(xx + dc).astype(int)
            rr = np.floor(yy + dr).astype(int)
            blocked = blocked or bool(np.any(self.dem[rr, cc] >= z - 1e-09))
```

```
    return (limit - free - (self.obs if blocked else 0), False)
```

问题三：中继服务日历调度关键代码

```
def decode(state, end, full=False):
    late = end + c[1]['back'] + 300 + c[2]['earliest_service'] + 10
    ur = {g: [(0, u) for u in pr.units if pr.units[u] == g] for g in 'ABC'}
    br = {g: [(0, b) for b, _ in pr.batts[g]] for g in 'ABC'}
    finishmax = energy = totalfinish = 0
    plan = []
    for g, ids in state:
        ut, u = min(ur[g])
        bt, b = min(br[g])
        earliest = max(ut, bt)
        opts = []
        for p, first, emin, emax, lmin, both, hard in profile(g, ids):
            start = max(earliest, c[0]['earliest_service'] + 6 - first, c[1]['
  earliest_service'] + 6 - emin, late + 6 - lmin, 0)
            upper = min(hard, end - 6 - emax)
            for _ in range(5):
                bad = [t for t in both if not (c[1]['earliest_service'] + 6 <=
  start + t <= end - 6 or start + t >= late + 6)]
                if not bad:
                    break
                start = max(start, late + 6 - min(bad))
            else:
                continue
            if start > upper + 1e-08:
                continue
            opts.append((start + p['finish'], p['energy'], start, p))
        if not opts:
            return None
        finish, en, start, p = min(opts, key=lambda x: x[:2])
        ready = finish + pr.charge(g, p['soc'])
        ur[g].remove((ut, u))
        ur[g].append((finish, u))
        br[g].remove((bt, b))
        br[g].append((ready, b))
        finishmax = max(finishmax, finish)
        energy += en
        totalfinish += finish
        if full:
            q = pr.evaluate(g, ids, p['route'], start)
            q.update(unit=u, battery=b, sortie=f'P{len(plan) + 1:03d}',
  charge_start=q['finish'], battery_ready=q['finish'] + pr.charge(g, q['soc']))
            plan.append(q)
    return plan if full else (finishmax, energy, totalfinish)
```

问题四：服务区分块与资源区间着色关键代码

```
def components(sites, edges):
    parent = {s: s for s in sites}

    def find(x):
        while parent[x] != x:
            x = parent[x]
        return x
    for edge in edges:
        for x in edge[1:]:
            parent[find(x)] = find(edge[0])
    groups = defaultdict(list)
```

```
    for s in sites:
        groups[find(s)].append(s)
    return sorted([sorted(v) for v in groups.values()])

def color(tasks, prefix):
    active = []
    free = []
    n = 0
    ans = []
    for t in sorted(tasks, key=lambda x: (x['start'], x['end'], x['task'])):
        while active and active[0][0] <= t['start']:
            _, i = heapq.heappop(active)
            heapq.heappush(free, i)
        if free:
            i = heapq.heappop(free)
        else:
            n += 1
            i = n
        heapq.heappush(active, (t['end'], i))
        ans.append(dict(t, resource=f'{prefix}-{i:02d}'))
    return (n, ans)
```

独立核验：四问事件回放入口

```
"""Replay into a scratch copy so Q3 frozen hashes remain unchanged."""
from pathlib import Path
import subprocess,sys,shutil,json,hashlib
ROOT=Path(__file__).resolve().parents[2];OUT=Path(__file__).parent;source='D:/研
  一/比赛/中文题目/D题';pre=ROOT/'outputs/preflight'
def run(script,*args):subprocess.run([sys.executable,str(script),*map(str,args)],
  check=True)
run(ROOT/'outputs/q2/verify_q2.py','--source',source,'--preflight',pre,'--out',
  OUT/'q2')
scratch=OUT/'replay_q3';scratch.mkdir(exist_ok=True)
shutil.copy2(OUT/'q3/summary.json',scratch/'summary.json')
for name in ['baseline','makespan_first']:
 d=scratch/name;d.mkdir(exist_ok=True)
 for file in ['plan.json','relay_plan.json','Q2_运输架次.csv','Q2_逐箱交付.csv']:
  shutil.copy2(OUT/'q3'/name/file,d/file)
run(ROOT/'outputs/q3/verify_joint.py','--source',source,'--preflight',pre,'--out
  ',scratch)
run(ROOT/'outputs/q4/verify_q4.py','--source',source,'--pre',pre,'--q3',OUT/'q3'
  ,'--out',OUT/'q4')
print('All independent replays PASS; frozen Q3 unchanged.')
```

独立核验：问题一往返能耗与逐箱检查

```
    def metrics(site,g,mass,count):
        r=models[g];o=original_nodes['O01'];s=original_nodes[site]
        dx=(s[2]-o[2])*geometry['meters_per_lon_degree'];dy=(s[3]-o[3])*geometry
  ['meters_per_lat_degree']
        distance=math.hypot(dx,dy)
        assert abs(distance-float(leg['O01',site]['horizontal_distance_m']))<1e-7
        cruise=D(leg['O01',site]['dem_max_m'])+50
        up1=cruise-D(o[4]);up2=cruise-D(s[4])-30
        q=D(mass);capacity=D(r[3]);v=q/capacity
        distance_range=D(r[6])-(D(r[6])-D(r[7]))*v*v.sqrt()
        horiz_go=D(r[8])*D(distance)/distance_range
        horiz_back=D(r[8])*D(distance)/D(r[6])
        climb_go=(D(r[2])+q)*D('9.80665')*up1/(D(r[16])*3600000)
        climb_back=D(r[2])*D('9.80665')*up2/(D(r[16])*3600000)
```

```
        e=horiz_go+horiz_back+climb_go+climb_back
        flight=(up1+up2)/D(r[14])+(up1+up2)/D(r[15])+2*D(distance)/D(r[5])
        operation=flight+D(r[10])+count*D(r[11])+D(r[12])+count*D(r[13])
        return e,flight,operation,1-e/D(r[8]),[horiz_go,climb_go,horiz_back,
climb_back]
```

```
    for sm in summaries:
        ps=[p for p in plans if p['scenario']==sm['scenario']];seen=Counter();
energies=[];times=[]
        for p in ps:
            bs=[boxes[b] for b in p['box_ids']];seen.update(p['box_ids'])
            assert all(b[1]==p['site'] for b in bs)
            q=sum(D(b[3]) for b in bs);v=sum(D(b[4]) for b in bs);m=models[p['
model']]
            assert q<=D(m[3]) and v<=D(m[4])
            e,tf,to,soc,parts=metrics(p['site'],p['model'],q,len(bs))
            assert soc+D('1e-10')>=D(p['rho'])
            assert abs(q-D(p['mass_kg']))<=D('1e-9') and abs(v-D(p['volume_m3']))
<=D('1e-12')
            assert abs(e-D(p['energy_kWh']))<=D('1e-8')
            assert abs(tf-D(p['flight_s']))<=D('1e-7') and abs(to-D(p['
operation_s']))<=D('1e-7')
            assert abs(soc-D(p['return_soc']))<=D('1e-10')
            maxerr=max(maxerr,float(abs(e-D(p['energy_kWh']))));energies.append(e
);times.append(to)
            details.append(dict(scenario=p['scenario'],sortie=p['sortie'],
outbound_horizontal_kWh=float(parts[0]),outbound_climb_kWh=float(parts[1]),
return_horizontal_kWh=float(parts[2]),return_climb_kWh=float(parts[3]),
energy_slack_kWh=float((1-D(p['rho']))*D(m[8])-e),status='PASS'))
        assert all(n==1 for n in seen.values())
        infeasible=[]
        for bid,b in boxes.items():
            available=False
            for g,m in models.items():
                if D(b[3])<=D(m[3]) and D(b[4])<=D(m[4]):
                    e,_,_,soc,_=metrics(b[1],g,b[3],1)
                    if soc>=D(sm['rho']):available=True
            if not available:infeasible.append(bid)
        assert set(infeasible)==set(filter(None,sm['infeasible_boxes'].split(';')
))
        if sm['status']!='INFEASIBLE':
            assert set(seen)==set(boxes) and len(ps)==sm['sorties']
            assert abs(sum(energies)-D(sm['energy_kWh']))<D('1e-8')
            assert abs(sum(times)-D(sm['operation_s']))<D('1e-7')
        else:
            assert infeasible
            expected={b for b,r in boxes.items() if r[1] not in sm['
infeasible_sites'].split(';')}
            assert set(seen)==expected
        checks.append(dict(scenario=sm['scenario'],checked_sorties=len(ps),
checked_boxes=len(seen),full_delivery=sm['status']!='INFEASIBLE',
confirmed_untransportable_boxes=len(infeasible),status='PASS'))
```

独立核验：问题二逐航段与硬时限检查

```
  bs=[boxes[b] for b in p['boxes']];seen.update(p['boxes'])
  q=sum(D(b[3]) for b in bs);volume=sum(D(b[4]) for b in bs)
  assert q<=D(m[3])+D('1e-9') and volume<=D(m[4])+D('1e-12')
  assert len(set(p['route']))==len(p['route']) and set(p['route'])=={b[1] for b
in bs}
  t=D(p['start'])+D(m[10])+len(bs)*D(m[11]);eq(t,p['takeoff']);energy=D(0);prev
='O01'
```

```
  assert len(p['legs'])==len(p['route'])+1
  for k,dest in enumerate(p['route']+['O01']):
   r=legs[prev,dest];a=nodes[prev];b=nodes[dest]
   dist=math.hypot((b[2]-a[2])*projection['meters_per_lon_degree'],(b[3]-a[3])*
 projection['meters_per_lat_degree'])
   eq(dist,r['horizontal_distance_m'])
   h=D(r['dem_max_m'])+50
   up=h-D(a[4])-(0 if prev=='O01' else 30);down=h-D(b[4])-(0 if dest=='O01' else
  30)
   x=q/D(m[3]);L=D(m[6])-(D(m[6])-D(m[7]))*x*x.sqrt()
   en=D(m[8])*D(dist)/L+(D(m[2])+q)*D('9.80665')*up/(D(m[16])*3600000)
   leg=p['legs'][k];assert leg['from_id']==prev and leg['to_id']==dest
   eq(t,leg['start']);eq(q,leg['payload']);eq(en,leg['energy'],'1e-8')
   t+=up/D(m[14])+D(dist)/D(m[5])+down/D(m[15]);eq(t,leg['end']);energy+=en
   if dest!='O01':
    here=[b for b in bs if b[1]==dest];stop=p['stops'][k];eq(t,stop['arrival'])
    t+=D(m[12])+len(here)*D(m[13]);eq(t,stop['complete'])
    assert set(stop['boxes'])=={b[0] for b in here}
    for b in here:
     eq(t,p['deliveries'][b[0]]);all_delivery[b[0]]=t
     limits=[]
     if b[2]=='医疗物资':assert b[7] is not None;limits.append(D(b[7]))
     if b[5]=='是':assert b[6] is not None;limits.append(D(b[6]))
     for deadline in limits:
      slack=deadline-t;hard_slacks.append(slack);assert slack>=D('-1e-7'),(b
 [0],float(slack))
    q-=sum(D(b[3]) for b in here)
   prev=dest
  assert abs(q)<D('1e-9');eq(t,p['finish']);eq(energy,p['energy'],'1e-8')
  soc=1-energy/D(m[8]);eq(soc,p['soc'],'1e-10');assert soc>=D(m[9])/100-D('1e
 -10')
  full=D(batteries[bat][1]);chg=full*(D('.65')*(D('.9')-soc)/D('.9')+D('.35'))
 if soc<D('.9') else full*D('.35')*(1-soc)/D('.1')
  eq(p['charge_start'],t);eq(p['battery_ready'],t+chg)
```

独立核验：问题二机体与电池事件回放

```
 assert set(seen)==set(boxes) and all(n==1 for n in seen.values())
 # Events release resources before new reservations at identical times.
machine={u:None for u in units};battery={b:'FULL' for b in batteries};trace=[]
for t,_,event,u,bat,flight in sorted(events):
 if event=='PREP_START':
  assert machine[u] is None and battery[bat]=='FULL',(t,u,bat,event)
  machine[u]=flight;battery[bat]=flight
 elif event=='LAND_AND_CHARGE':
  assert machine[u]==flight and battery[bat]==flight
  machine[u]=None;battery[bat]='CHARGING'
 else:
  assert battery[bat]=='CHARGING';battery[bat]='FULL'
 trace.append(dict(time_s=t,event=event,unit=u,battery=bat,sortie=flight,
unit_state=machine[u] or 'AVAILABLE',battery_state=battery[bat]))
delay=sum(D(boxes[b][8])*max(D(0),t-D(boxes[b][7])) for b,t in all_delivery.
items() if boxes[b][7] is not None)
eq(delay,summary[name]['D']);eq(maxT,summary[name]['Cmax']);eq(totalE,summary[
name]['E'],'1e-8');assert len(plan)==summary[name]['N']
```

独立核验：问题三通信状态与切换检查

```
def scan(self,phases,relays,dt,spacing,detail=False):
 back={r['sortie']:self.margin(np.array(r['position']),self.gateway,'backhaul',
 spacing) for r in relays};rows=[];coverage=[];transitions=[];worst=None;bad=[];
```

```
n=0
 for ph in phases:
  a=np.array(ph['a']);b=np.array(ph['b']);start,end=ph['start'],ph['end']
  def rawstate(t):
   pos=a+(b-a)*(t-start)/(end-start);direct=self.margin(pos,self.gateway,'
direct',spacing)
   if direct>=-1e-9:return 'G01',direct,direct,pos
   avail=[(min(self.margin(pos,np.array(r['position']),'access',spacing),back[r
['sortie']]),r['sortie']) for r in relays if r['service_start']-1e-8<=t<=r['
service_end']+1e-8]
   m,provider=max(avail,default=(-999,'NONE'));return provider if m>=-1e-9 else
'NONE',m,direct,pos
  memo={}
  def state(t):
   if t not in memo:memo[t]=rawstate(t)
   return memo[t]
  times=set(np.linspace(start,end,max(1,math.ceil((end-start)/dt))+1).tolist())
  times.update(t for r in relays for t in [r['service_start'],r['service_end']]
 if start<t<end)
  if detail:
   base=sorted(times)
   for left,right in zip(base,base[1:]):
    sl=state(left);sr=state(right)
    if min(sl[1],sr[1])<.02 or sl[0]!=sr[0]:
     times.update(np.linspace(left,right,max(1,math.ceil((right-left)/.05))+1).
tolist())
  prev=None;segment=start
  for t in sorted(times):
   provider,m,direct,pos=state(t);n+=1
   if provider=='NONE':bad.append(dict(sortie=ph['sortie'],phase=ph['phase'],
time=t,margin=m))
   if worst is None or m<worst['margin_db']:worst=dict(sortie=ph['sortie'],
phase=ph['phase'],time_s=t,margin_db=m,provider=provider)
   if detail:
    rows.append(dict(sortie=ph['sortie'],phase=ph['phase'],time_s=t,x_m=pos[0],
y_m=pos[1],z_m=pos[2],provider=provider,margin_db=m,direct_margin_db=direct))
    if prev and provider!=prev[1]:
     lo=prev[0];hi=t
     for _ in range(30):
      if hi-lo<1e-7:break
      mid=(lo+hi)/2
      if state(mid)[0]==prev[1]:lo=mid
      else:hi=mid
     cut=(lo+hi)/2;transitions.append(dict(sortie=ph['sortie'],phase=ph['phase'
],lo=lo,hi=hi,old=prev[1],new=provider))
     coverage.append(dict(sortie=ph['sortie'],phase=ph['phase'],start=segment,
end=cut,provider=prev[1],site=ph['site'],from_site=ph['from_site'],to_site=ph[
'to_site'],associated_services=ph['associated_services']));segment=cut
   prev=(t,provider)
  if detail:coverage.append(dict(sortie=ph['sortie'],phase=ph['phase'],start=
segment,end=end,provider=prev[1],site=ph['site'],from_site=ph['from_site'],
to_site=ph['to_site'],associated_services=ph['associated_services']))
 return dict(dt_s=dt,spatial_spacing_m=spacing if spacing else 'all crossed
cells and touched corners',points=n,failed_points=len(bad),worst=worst,claim='
在所述时空分辨率下未发现中断' if not bad else '发现通信中断采样点'),rows,coverage,
transitions,bad
```

独立核验：问题四冻结任务与资源配置检查

```
d=a.out/name;groups=load(d/'groups.json');tasks=load(d/'tasks.json');alloc=read(d/'
resource_assignment.csv');coverage=read(d/'communication_inheritance.csv');
all_sites=[v for g in groups for v in g['sites']];assert sorted(all_sites)==sorted(
```

```
 nodes.keys()-{'O01'}) and len(all_sites)==15 and all(g['sites'] for g in groups);
groupof={site:g['group'] for g in groups for site in g['sites']}
 for x in p:assert len({groupof[site] for site in x['route']})==1
 expected_relay={(groupof[ps[x['运输架次编号']]['route'][0]],x['中继架次编号']) for x in
comm if x['中继架次编号'] and float(x['结束时刻（s）'])>float(x['开始时刻（s）'])}
 assert {(x['group'],x['original_task']) for x in tasks if x['kind']=='relay'}==
expected_relay
 assert len([x for x in tasks if x['kind']=='transport'])==len(p) and len(tasks)==len
(p)+len(expected_relay)
 assert len({(x['group'],x['original_task']) for x in tasks})==len(tasks)
 for x in tasks:
  assert x['frozen']==original[x['original_task']]
  if x['kind']=='transport':assert x['group']==groupof[x['frozen']['route'][0]]
 expected_alloc=set()
 for x in tasks:
  g=x['frozen']['model'] if x['kind']=='transport' else 'R'
  for typ,rid in [(g+'_drone',x['unit']),('R_energy' if g=='R' else g+'_battery',x['
energy_component'])]:expected_alloc.add((x['group'],x['original_task'],typ,rid))
 assert len(alloc)==len(expected_alloc) and {(x['group'],x['task'],x['type'],x['
resource']) for x in alloc}==expected_alloc
 template=read(d/'Q4_分区配置.csv');headers=[h for h in list(load_workbook(find('结果提
交模板.xlsx'),data_only=True)['Q4_分区配置'].values)[0] if h];assert list(template[0])
==headers
 for row,g in zip(template,groups):
  assert int(row[headers[0]])==len(groups) and row[headers[1]]==g['group'] and row[
headers[2]].split(';')==g['sites'];assert [int(row[h]) for h in headers[3:]]==g['
resources']
 assert len(coverage)==len(comm)
 sortkey=lambda x:(x['运输架次编号'],float(x['开始时刻（s）']))
 for x,y in zip(sorted(coverage,key=sortkey),sorted(comm,key=sortkey)):
  assert all(x[k]==v for k,v in y.items());assert x['group']==groupof[ps[y['运输架次编
号']]['route'][0]]
  assert x['relay_copy']==(x['group']+'-'+y['中继架次编号'] if y['中继架次编号'] else '')
 owner={};ev=[];peaks=[]
 for x in alloc:
  src=original[x['task']];idx=0 if x['type'].endswith('drone') else 1;eq(x['start'],
src['start']);eq(x['end'],releases[x['task']][idx]);assert x['resource'] not in
owner or owner[x['resource']]==x['group'];owner[x['resource']]=x['group'];ev += [(
float(x['start']),1,x),(float(x['end']),0,x)]
 state={};trace=[]
 for t,kind,x in sorted(ev,key=lambda e:(e[0],e[1],e[2]['resource'])):
  rid=x['resource']
  if kind:assert rid not in state;(state.update({rid:x['task']}))
  else:assert state.pop(rid)==x['task']
  trace.append(dict(time_s=t,event='RESERVE' if kind else 'READY',resource=rid,group
=x['group'],task=x['task'],occupied_resources=len(state)))
 assert not state;write(d/'independent_event_replay.csv',trace)
 W=[];tot=np.zeros(8,dtype=int)
 for g in groups:
  ww=0.
  for i,typ in enumerate(types):
   seq=[x for x in alloc if x['group']==g['group'] and x['type']==typ];peak=max([sum(
float(x['start'])<=float(t['start'])<float(x['end']) for x in seq) for t in seq],
default=0);assigned=len({x['resource'] for x in seq});assert assigned==g['resources'
][i];tot[i]+=assigned
   if result['mode']=='renumber':assert peak==assigned
   else:assert assigned==len({x['original_resource'] for x in seq})
   if typ.endswith('drone'):ww+=math.fsum(float(x['end'])-float(x['start']) for x in
seq)
  eq(ww,g['work_s']);W.append(ww)
 assert tot.tolist()==result['total_resources'];G=sum(max(0,int(x)-cap)/cap for x,cap
 in zip(tot,inventory));S=sum(int(x)/cap for x,cap in zip(tot,inventory));cv=math.
sqrt(max(0,len(W)*sum(w*w for w in W)/sum(W)**2-1));eq(G,result['G']);eq(S,result['S
']);eq(cv,result['CV'],1e-8)
```